FINASTRA

Lending

TRADE INNOVATION 2.11

SDK - Systems Interfacing Guide

Version 1.0.1

Finastra Technical Publications
November 2025

INNOVATING
FINANCE
TOGETHER

Version History

Date	Version	Change Reference
29/05/2025	1.0	Initial Version
28/11/2025	1.0.1	Minor updates under Pseudo-Synchronous JMS

CONTENTS

VERSION HISTORY ..... **II**
PREFACE ..... **VI**
Screen Examples vi
Document Conventions vi
Further Reading vii
OVERVIEW **1**
Messaging Services Interfaces in Trade Innovation 1
Service Interface Architecture 1
Interfacing 3
Customer Gateway Interface Service 3
MESSAGE STRUCTURE **5**
Common Protocol Details 5
Request/Response 6
Request/DeferredResponse 9
Bulk Service Support 11
Batch Schemas 13
Schema Files 13
Naming Convention 14
Namespaces 14
Verifying Message Formats 15
EJB TRANSPORT SERVER COMPONENT **18**
GLOBAL JMS TRANSPORT SERVER **19**
JMS COMPONENTS **20**
Plain JMS Components 20
Transport Server 20
Pseudo-Synchronous JMS 20
Transport Server 20
Transport Client 21
Message Transformation 21
Trade Innovation Corporate Channels Interface 22
Transport Server 22
Transport Client 22
MT798 Interface 22
Transport Server 22
Transport Client 22
Equation Interface Support 22
Watch List Check Interface Support 22
SERVICE PROVIDER INTERFACES (SPIS) **23**

APISTUBS **25**
Transport Server 25
Transport Client 25
The Data Editor 25
Installing the Data Editor 25
Using the Data Editor 27
DEVELOPING COMPONENTS **31**
Transport Client Implementation 31
Configuring the Transport Client 31
Testing 32
Available Balance Request 33
Available Balance Response 33
Developing a Server Interface 33
Message Capture 34
Message Transformation 35
Calling the ServiceProvider 35
SPECIFIC MESSAGE PROCESSING **38**
Accounts 38
Search 38
Retrieve 38
Open 38
Balance Checking 38
Customers 39
Search 39
Retrieve 39
Business Support 39
Exchange Rates 39
Interest Rates 39
Payment 39
Foreign Exchange Contracts 39
Back Office 40
Foreign Exchange Deals 40
Postings 41
Charge Details 41
Verify 42
Limits 42
Facilities 42
Credit Risk 43
Gateway 43
Notifications 43
Event Step Status Payload 43
Master Archive Deletion Payload 44

Job Status Change Payload 45
Step Status Change Payload 45
SWIFT 46
Outgoing Messages 46
BIC Files 46
TI 46
Controlling End of Day 46
Transaction Event Messages 47
Enquiry Services 48
Global 48
WEB SERVICES 49
Imaging Support 49
External DMS 49
APPENDIX A MESSAGE SCHEMAS 51
Synchronous Messages 51
Asynchronous Messages 52
Miscellaneous TI Messages 52
TI Enquiry Messages 52
TI Transaction Messages 53
Static Data/System Tailoring/Security Messages 58
Global Messages 62
APPENDIX B THE APISTUBS DATA MODEL 63
Table - APIServer 63
Purpose 63
Details 63
Table - APIResponse 64
Purpose 64
Details 64
Table - APIClient 64
Purpose 64
Details 64
APPENDIX C CONFIGURING WATCHLIST CHECKER REPEATING FIELDS 65
Repeating Fields 65
Defining Field Sets 65
Configuring Repeating fields for Watchlist Checking 66
Sanction Screening SPI (Watchlist checker) 66
Limitations 67
Repeating Fields should have consistent Field Count 67

Preface

This Guide describes how messages are passed between Trade Innovation and third party systems. Since the nature of these third party systems can vary, this Guide concentrates on the use of the supplied technical infrastructure. It is intended to be read by technical staff, responsible for implementing interfaces to Trade Innovation systems.

This is not an end-user task and so this document is targeted at technical consultants and bank staff. It assumes that you are familiar with the basic principles of trade finance and with Trade Innovation itself. Experience of programming is a requirement for understanding the more technical parts of the document.

The User Guide contains the following chapters:

Chapter	Description
Overview	Provides an overview of Trade Innovation interface services and processing.
Message Structure	Provides details of common protocol details for request and response messages.
EJB Transport Server Component	Provides details of the EJB transport server component.
Global JMS Transport Server	Provides details of the JMS interface components.
JMS Components	Provides details about JMS interface components supplied.
APIStubs	Provides details of the APIStubs testing utility provided.
Developing Components	Provides information on developing a both outgoing and incoming interface components.
Specific Message Processing	Provides information on some of the messages and processing used by Trade Innovation to communicate with your bank's core banking system.
Web Services	Provides information on the web services Trade Innovation can optionally host and use.
Appendix A	Provides lists of the message schemas used by Trade Innovation Plus.
Appendix B	Provides details on the APIStubs data model.

Screen Examples

This guide includes examples of screens. The screens shown in your system may not look the same as the examples in this guide. However the actual content and user operation are identical

Document Conventions

The convention used in the Guide for identifying links accessed via drop-down lists from other links is to cite the name of the first link, followed by name of the second link separated from it by a '|' character - for example, ‘Other|FX Calculator'.

Note: This format is used for notes that contain information of more than usual significance, such as warnings or hints on using the software.

✓ In the tables listing fields in windows, a tick next to a field indicates that the field is mandatory.

Further Reading

Trade Innovation is supported by a comprehensive documentation set, which includes user guides for each Trade Innovation product:

- *Documentation Overview - Trade Innovation* lists each of the documents in the set and explains what it covers.
- *Common Facilities User Guide - Trade Innovation* explains common screen elements shared by Trade Innovation products and is referred to where required in this guide.

Overview

This chapter provides an overview of Trade Innovation interface services and their processing.

Messaging Services Interfaces in Trade Innovation

Trade Innovation has been designed to operate within a service-oriented architecture. Trade Innovation functions as both a *provider* of services, where external systems make requests of Trade Innovation, and as a *consumer* of services, where Trade Innovation makes requests of external systems. These requests can be *synchronous*, where the consuming system waits for a response from the providing service, or *asynchronous*, where the consuming system does not wait for such a response. For asynchronous requests the response is received as an entirely separate service call from the provider.

The messages described in this Guide are of two types:

- *Core banking interface services*, a set of which is delivered with Trade Innovation. The messages used with these services have a fixed layout. They make and respond to requests both synchronously and asynchronously, and are designed to allow Trade Innovation to share information with your bank's core banking system - for example, to perform real-time balance checks. Your bank will need to develop functionality to allow Trade Innovation and your core banking system to communicate with each other using these messages
- *Customer gateway interface service*, which are used to receive information from your customers, and to send messages to them, via your bank's customer access system. These messages are processed asynchronously. A set of customer gateway interface service messages is delivered with Trade Innovation. The layouts for incoming request messages are fixed, but your bank can define its own layouts for outward request messages.

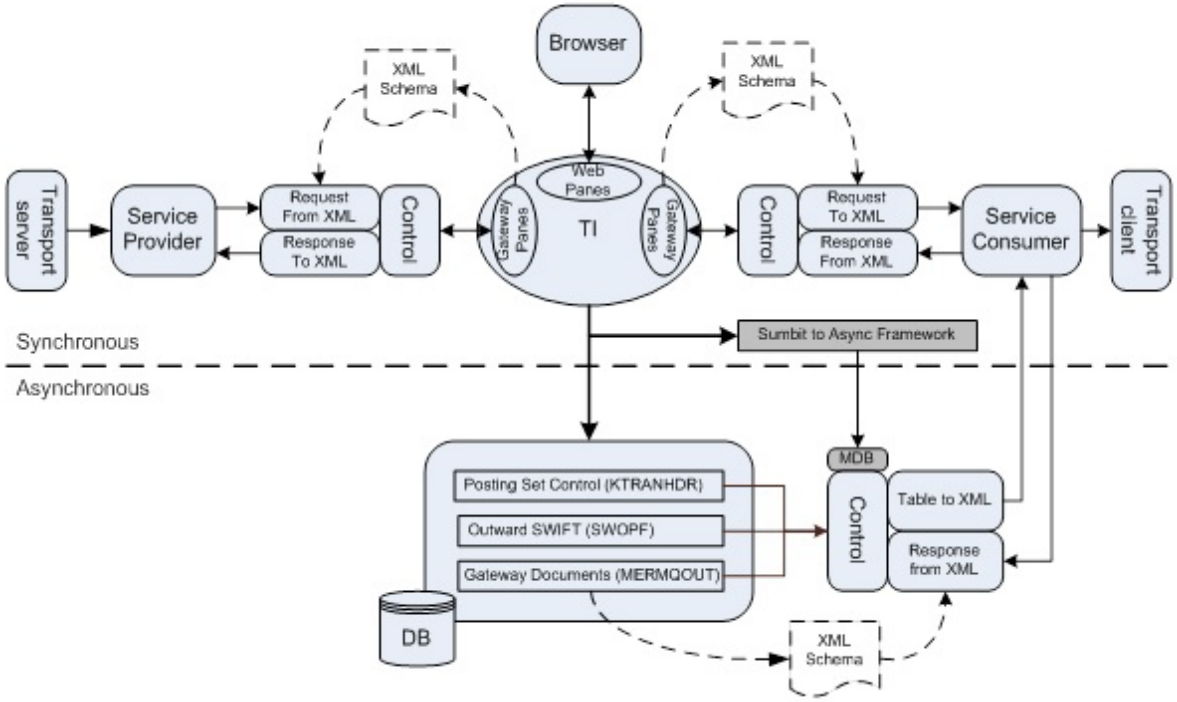
Service Interface Architecture

Service calls and responses are effected using XML messages passed through a messaging infrastructure. Development work is required at your bank to handle these incoming and outgoing XML messages, transforming them into messages that the external systems can support, and delivering them over the networking infrastructure in use at your bank. This may involve developing Java components to be plugged in to the Trade Innovation messaging infrastructure or using one of the supplied components.

In a multi-zone environment, each zone supports its own messaging infrastructure, allowing the zones to be connected to entirely different external systems.

In a Global Processing environment, each zone may be connected to multiple external systems, each of which performs the same function for different parts of the branch hierarchy. Routing information is available to a mapping/routing layer in your messaging infrastructure. It is also possible to use the identifier of the external system as part of the configuration as a differentiator.

The following diagram illustrates the infrastructure of the service processing in Trade Innovation:



All service operations are invoked using an XML request message, with an XML response message being returned. Each service operation consists of a specific request message and usually a specific response message, in addition to common protocol details. XML message formats take the form of xsd schema files, with a separate schema file for each type of request and response.

Asynchronous service requests originating from Trade Innovation are queued on the zone database. A control process then forwards them to the service infrastructure. This means that Trade Innovation processing can continue without waiting for a response from the external system. Asynchronous service calls are expected to succeed, since the data being transmitted will have been validated as part of the Trade Innovation workflow before it was sent. If the service call cannot reach the target system, the service call can be resent, but any repair to the message data will need to be carried out in the external system.

Asynchronous service requests fall into two categories:

- Those that expect an immediate response, as they are simply transferring information to the external system
- Those that expect a deferred response from an external system

A deferred response is one that is an incoming message that matches an outgoing request. When processed, an incoming deferred response returns an acknowledgment.

Based on the request header directives the infrastructure will treat incoming requests synchronously or asynchronously. If a more sophisticated asynchronous process is required, it must be provided in the interface layer.

Interfacing

Where your bank wishes to supply its own interfacing components, this is done by implementing *transport servers* that receive service requests and pass them into Trade Innovation, *transport clients* that receive service requests from Trade Innovation and pass them to the external infrastructure or for a some of the outgoing APIs from Trade Innovation, there is an option to implement a REST endpoint, known as a Service Provider Interface (SPI).

Transport Servers

All incoming service requests must use a transport server as an entry point. Each server will have its own rules of how it must be invoked.

All transport servers interface to the same service providers which implement the `ServiceProvider` interface. These are components that are resolved based on the name of the service to use.

Transport Clients

A transport client receives a service request for an external service. It must implement the `TransportClient` interface, and be configured against the relevant interface service operations.

All transport clients must return a response immediately. If the call to an external service is made through an asynchronous transport mechanism, then it is the responsibility of the transport client to manage the response accordingly. For a request that does not require a functional response, a successful response returned by the transport client can indicate that the request was successfully sent to the external service.

For service requests that require an immediate functional response, the transport client must block until that response is returned.

For service requests that require a deferred functional response, an immediate response is required to indicate that the request is being delivered to the external system.

Customer Gateway Interface Service

The term *customer gateway interface service message* is used to cover messages passed between Trade Innovation and your bank's customer access system (Finastra Corporate Channel, for example). In general, these messages are used to initiate new events in Trade Innovation and to inform a gateway customer of the outcome of events initiated through the gateway by that customer, or by another party to the transaction, such as an overseas bank.

Inward messages conform to the published schemas, but outgoing messages are created using the facilities provided by Trade Innovation for defining outward documents. You can set up document types and templates for each type of outward customer gateway interface service message and define which event fields are to provide the information it will contain. Trade Innovation determines when an outgoing message is to be sent out through the customer gateway interface service, either from the document being created or from the addressee who will receive the message.

The interface has been designed to support messages for the following Trade Innovation products:

- Clean payments
- Export letters of credit
- Export guarantees
- Export standby letters of credit
- Factoring facility
- Finance Standalone
- Import letters of credit
- Import guarantees

- Import standby letters of credit
- Invoices
- Outward collection orders
- Licenses

However, you can set up documents to be sent via the customer gateway interface service for all events, not just for those belonging to the products supported by the customer gateway interface service. Where a document addressee is a customer gateway interface service user, then the document will be sent via the customer gateway interface service upon release of the event.

See the *System Tailoring User Guide - Trade Innovation* for instructions on setting up document types and layouts and the *Static Data Maintenance User Guide – Trade Innovation* for information on setting up customers as customer gateway interface service users.

Message Structure

This chapter covers the common structure of the request and response messages used by interface services.

Common Protocol Details

Each service call is wrapped in a common messaging protocol. The common protocol relates to the facilities provided and expected by Trade Innovation.

The protocol facilities are as follows:

Section	Description
Security	The credentials supplied are used to make sure the service use is authorised.
Repair control	When errors or warnings are generated, the protocol provides directives set by the service client indicating whether the client or the service is responsible for repairing the data.
Required response	The service client may or may not be interested in the response of a service operation (particularly relevant if repair is expected to be controlled by the service operation).
Response matching	The service client may not require response matching as it may block while waiting for the response; however, if the client is queue-based, it would need to match responses to requests.

The protocol data wraps the functional request or response data. All request messages are `ServiceRequest` or `DeferredResponse` elements, and the associated response messages are `ServiceResponse` or `Aknowledgement` elements.

Where a response is synchronous, a `ServiceResponse` is returned. Where a response is asynchronous, an immediate `ServiceResponse` is expected, followed by a subsequent `DeferredResponse` being sent. An `Acknowledgement` is then returned.

These are documented in the following sections.

Request/Response

ServiceRequest

```
<ServiceRequest>
    <RequestHeader>
        <Service>XXXX</Service>
        <Operation>XXXX</Operation>
        <Credentials>
            <User>XXXX</User>
            <Password>XXXXXXXX</Password>
            <Certificate>XXXXX</Certificate>
            <Digest>XXXX</Digest>
        </Credentials>
        <ReplyFormat>XXXX</ReplyFormat>
        <ReplyTarget>XXXX</ReplyTarget>
        <TargetSystem>XXXX</TargetSystem>
        <SourceSystem>XXXX</SourceSystem>
        <NoRepair>X</NoRepeair>
        <NoOverride>X</NoOverride>
        <CorrelationId>XXXXX</CorrelationId>
        <TransactionControl>XXXXX</TransactionControl>
        <CreationDate>XXXXX</CreationDate>
        <GroupingId>XXXXX</GroupingId>
    </RequestHeader>
    <Request*/>
</ServiceRequest>
```

The `Request` element is defined as abstract, and used as a substitution group for the functional data.

Element		Description	
Service		The name of service to interact with.	
Operation		Operation provided by the service.	
Credentials		A structured type that contains the following user credentials.	
	User	The user name.	
	Password	Reserved for future use The user's password (optional, depending on security configuration), usually encrypted.	
	Certificate	Reserved for future use X509 certificate.	
	Digest	Reserved for future use The message digest.	
ReplyFormat		Indicates the form of response expected by the client:	
		NONE	No reply is expected – with this option, the message will be processed though the async framework.
		STATUS	The client expects the protocol response header.
		FULL	The client expects the protocol response header and functional response data, if applicable.
ReplyTarget		For transport protocols that require a response, this can contain a JNDI name and will be used if a reply target cannot be inferred.	

Element	Description	
TargetSystem	Identifier of system to route the request to from – this will need to be set to the zone id of the target instance of Trade Innovation for incoming requests.	
SourceSystem	Identifier of system the request came from – this will be set to the zone id of the originating instance of Trade Innovation for outgoing requests.	
NoRepair	If set to 'Y', and Trade Innovation provides a repair mechanism for this transaction type, then if an error occurs, the message is returned rather than being repaired. The default setting is N, with Trade Innovation repairing the message.	
NoOverride	If set to 'Y' and warnings are generated in Trade Innovation, then the transaction is not accepted and the warnings are returned. The default setting is 'N', with warnings associated with the transaction being processed by a user within Trade Innovation.	
CorrelationId	The request ID. If supplied, this must be returned in the response protocol header.	
TransactionControl	Indicates where the transaction control is to be used:	
	NEW	Reserved for future use. An explicit transaction control is dictated.
	NONE	Control is with Trade Innovation.
	INHERIT	Reserved for future use. Transaction processing needs to be attached to the transaction in progress and not commit or rollback, as the caller is in control.
CreationDate	Holds the business date associated with the data on the message.	
GroupingId	Allows a common reference to be associated with a set of messages that can be searched for in the Incoming gateway message manager browser.	

ServiceResponse

```
<ServiceResponse>
    <ResponseHeader>
        <Service>XXXX</Service>
        <Operation>XXXX</Operation>
        <Status>XXXX</Status>
        <Details>
            <Error>xxxxxxx</Error>
            <Warning>xxxxxx</Warning>
            <Info>xxxxx</Info>
        </Details>
        <CorrelationId>XXXXX</CorrelationId>
        <TargetSystem>XXXX</TargetSystem>
        <SourceSystem>XXXX</SourceSystem>
    </ResponseHeader>
    <Response*/>
</ServiceResponse>
```

The `Response` element is defined as abstract, and used as a substitution group for the returned functional data.

Element		Description	
Service		The name of service to interact with.	
Operation		Operation provided by the service.	
Status		Indicates the status of the request:	
		SUCCEEDED	The request has been successful. Note that this does not mean that the request was functionally correct. If it was not, errors will be returned in the details section. For incoming service requests if the message had errors and was repaired in Trade Innovation, the status response is set to SUCCEEDED, and an information message may be supplied indicating that the message has been placed in a repair queue.
		FAILED	The request has failed, and no updates associated with the request will have been made. Any errors and warning messages are expected to be returned, together with any information messages.
		UNAVAILABLE	The request was not processed due to system unavailability. Any information messages may also be returned.
Details		Messages generated by service operation – multiple messages of each type may be returned.	
	Error	The text of any error messages.	
	Warning	The text of any warning messages.	
	Info	The text of any information messages.	
CorrelationId		The request ID – note that this is not a JMS correlation id.	
TargetSystem		Identifier of system to route the response to – this will need to be set to the zone id of the target instance of Trade Innovation for incoming response.	
SourceSystem		Identifier of system the response came from – this will be set to the zone id of the originating instance of Trade Innovation for outgoing response.	

Request/DeferredResponse

In this pattern, there are two sets of messages:

- Initial outgoing `ServiceRequest` with a `ServiceResponse` acknowledging that the request has been successfully passed to the target system (or at least the intermediate middleware)
- Incoming `DeferredResponse` that has the `CorrelationId` of the outgoing `ServiceRequest`, and will be matched based on that value. Once processed, an `Acknowledgment` message is returned.

DeferredResponse

```
<DeferredResponse>
    <DeferredHeader>
        <Service>XXXX</Service>
        <Operation>XXXX</Operation>
        <Credentials>
            <User>XXXX</User>
            <Password>XXXXXXXX</Password>
            <Certificate>XXXXX</Certificate>
            <Digest>XXXX</Digest>
        </Credentials>
        <ReplyFormat>XXXX</ReplyFormat>
        <ReplyTarget>XXXX</ReplyTarget>
        <Status>XXXX</Status>
        <Details>
            <Error>xxxxxxx</Error>
            <Warning>xxxxxx</Warning>
            <Info>xxxxx</Info>
        </Details>

<TargetSystem>XXXX</TargetSystem>
        <SourceSystem>XXXX</SourceSystem>
        <CorrelationId>XXXXX</CorrelationId>
        <TransactionControl>XXXXX</TransactionControl>
        <CreationDate>XXXXX</CreationDate>
    </DeferredHeader>
    <Request*/> or <Response*/>
</DeferredResponse>
```

The `Request` or `Response` elements are defined as abstract, and used as a substitution group for the functional data.

Element		Description	
Service		The name of service to interact with.	
Operation		Operation provided by the service.	
Credentials		A structured type that contains the following user credentials.	
	User	The user name.	
	Password	Reserved for future use. The user's password (optional, depending on security configuration), usually encrypted.	
	Certificate	Reserved for future use. X509 certificate.	
	Digest	Reserved for future use. The message digest.	
ReplyFormat		Indicates the form of response expected by the client:	
		NONE	No reply is expected – with this option, the message will be processed though the async framework.
		STATUS	The client expects the protocol response header.
		FULL	The client expects the protocol response header and functional response data, if applicable.
ReplyTarget		For transport protocols that require a response, this can contain a JNDI name and will be used if a reply target cannot be inferred.	
Status		Indicates the status of the request:	
		SUCCEEDED	The request has been successful. Note that this does not mean that the request was functionally correct. If it was not, errors will be returned in the details section. For incoming service requests if the message had errors and was repaired in Trade Innovation, the status response is set to SUCCEEDED, and an information message may be supplied indicating that the message has been placed in a repair queue.
		FAILED	The request has failed, and no updates associated with the request will have been made. Any errors and warning messages are expected to be returned, together with any information messages.
Details		Messages generated by service operation – multiple messages of each type may be returned.	
	Error	The text of any error messages.	
	Warning	The text of any warning messages.	
	Info	The text of any information messages.	
TargetSystem		Identifier of system to route the request to from – this will need to be set to the zone id of the target instance of Trade Innovation for incoming requests.	
SourceSystem		Identifier of system the request came from – this will be set to the zone id of the originating instance of Trade Innovation for outgoing requests.	
CorrelationId		The request ID. If supplied, this must be returned in the response protocol header.	

Element	Description	
TransactionControl	Indicates where the transaction control is to be used:	
	NEW	Reserved for future use. An explicit transaction control is dictated.
	NONE	Control is with Trade Innovation.
	INHERIT	Reserved for future use. Transaction processing needs to be attached to the transaction in progress and not commit or rollback, as the caller is in control.
CreationDate	Holds the business date associated with the data on the message.	

While the Status field can take the value `UNAVAILABLE`, since this is an asynchronous protocol, it makes more sense to return an `ERROR` status and an error message.

Acknowledgment

```
<<Acknowledgment>
    <AcknowledgmentHeader>
        <Service>XXXX</Service>
        <Operation>XXXX</Operation>
        <Status>XXXX</Status>
        <Message>xxxxxxx</Message>
        <CorrelationId>XXXXX</CorrelationId>
        <TargetSystem>XXXX</TargetSystem>
        <SourceSystem>XXXX</SourceSystem>
    </AcknowledgmentHeader>
</Acknowledgment>
```

Element	Description	
Service	The name of service.	
Operation	Operation provided by the service.	
Status	Indicates the status of the request:	
	SUCCEEDED	The request has been successful.
	FAILED	The request has failed, and no associated updates will have been made. Details are supplied in the <Message> element.
	UNAVAILABLE	The request was not processed due to system unavailability. An information message may also be returned.
Message	Message associated with FAILED status.	
CorrelationId	The request ID – note that this is not a JMS correlation id.	
TargetSystem	Identifier of system to route the acknowledgement to.	
SourceSystem	Identifier of system the acknowledgement came from – this will be set to the zone id.	

Bulk Service Support

A bulk service is available to allow multiple services to be invoked. The bulk service request message (including the common protocol details) is structured as follows:

```
<ServiceRequest>
    <RequestHeader>...</RequestHeader>
```

```
    <ItemRequest>
        <ServiceRequest>
            <RequestHeader>...</RequestHeader>
            <Request...
        </ServiceRequest>
        <ServiceRequest>
            <RequestHeader>...</RequestHeader>
            <Request...
        </ServiceRequest>
    </ItemRequest>
    <ItemRequest...
</ServiceRequest>
```

For incoming service requests, the service for this message is `TIBulk` and the operation is `Item`. Embedded `ServiceRequest`s each have their individual service and operations specified.

The response is structured similarly:

```
<ServiceResponse>
    <ResponseHeader>...</ResponseHeader>
    <ItemResponse>
        <ServiceResponse>
            <ResponseHeader>...</ResponseHeader>
            <Response...
        </ServiceResponse>
        <ServiceResponse>
            <ResponseHeader>...</ResponseHeader>
            <Response...
        </ServiceResponse>
    </ItemResponse>
    <ItemResponse...
</ServiceResponse>
```

When received, if the message is to be processed asynchronously, the individual `ServiceRequest`s of each `ItemRequest` will be processed in parallel with each other, while each `ItemRequest` is processed after the previous one until all are processed. This means that if one `ServiceRequest` depends on another being processed first then they must be in separate `ItemRequest` sections.

Once they are all complete, the combined response is returned. In this case, the `RequestHeader`s of the individual messages must be defined as if the message was being sent on its own.

If the message is processed synchronously, the `RequestHeader` of the bulk message is merged with each of the individual messages as they are processed.

In the case of the `GroupingId` tag, the individual message value takes precedence over the value at the bulk message level. If the individual message does not specify a `GroupingId`, then the one specified on the bulk message is used.

Note: This message structure is also used when passing account posting sets asynchronously as `BackOffice.Batch` and synchronously as `BackOffice.Verify`.

Maximum size limit of TIBulk message(s) is 130800 characters.

Batch Schemas

Some batch export files may be in XML format. Typically, they will re-use existing schema definitions for the individual items, where the batch schema defines the wrapper.

A common BatchHeader is used to represent the whole set of items:

```
<BatchHeader>
    <Batch>XXXX</Batch>
    <SourceSystem>XXXX</SourceSystem>
    <TargetSystem>XXXX</TargetSystem>
    <ProcessingCycle>XXXX</ProcessingCycle>
    <ProcessingDate>XXXX</ProcessingDate>
    <MainBankingEntity>xxxxxxx</MainBankingEntity>
    <CorrelationId>XXXXX</CorrelationId>
</BatchHeader>
```

Element	Description
Batch	The name of service.
SourceSystem	Identifier of system the acknowledgement came from – this will be set to the zone id.
TargetSystem	Identifier of system to route the acknowledgement to.
ProcessingCycle	The name of the processing cycle the export file was produced in
ProcessingDate	The business date when the export was generated
MainBankingEntity	The main banking entity of the export contents
CorrelationId	The request ID – note that this is not a JMS correlation id.

For example, the Postings export file has a batch header and a repeating set of PostingItem's, each of which contain a Posting definition based on the format defined for the BackOffice.Posting request.

Schema Files

The schema definitions are provided in a two jar files that are also included on the classpath of the application so that messages received by Trade Innovation are validated prior to processing their contents:

com.misys.tiplus2.gen.ti-xsd.jar

com.misys.tiplus2.gen.ti-xsd.customisation.jar

The content can be extracted using a standard zip extract utility.

The jar files have the following folder structure:

`/batch`		Contains the batch wrapper definitions for inclusion of existing service request formats.
`/control`		Contains the `ServiceRequest` and `ServiceResponse` schema files.
`/services`		Contains a schema for each service operation. Includes the request message of the service call and if appropriate the response message.
	`/common`	Contains a schema file for common data types.

	/custom	Contains a schema file for extra data data types. This is held in the customisation jar file.
	/messages	Contains a schema file for each individual request and response message.

Naming Convention

A service operation is made up of a service identifier and an operation identifier. If the service operation provides an immediate functional response, then the names of the messages are:

<operation>Request

and

<operation>Response

respectively.

If the service operation only provides the common protocol response, then the name of the request message is:

<operation>

Note: A response to this sort of message is a `ServiceResponse` without a functional payload, or an `Acknowledgment` if it came in as a `DeferredResponse`.

Namespaces

The following namespaces used within a message definition:

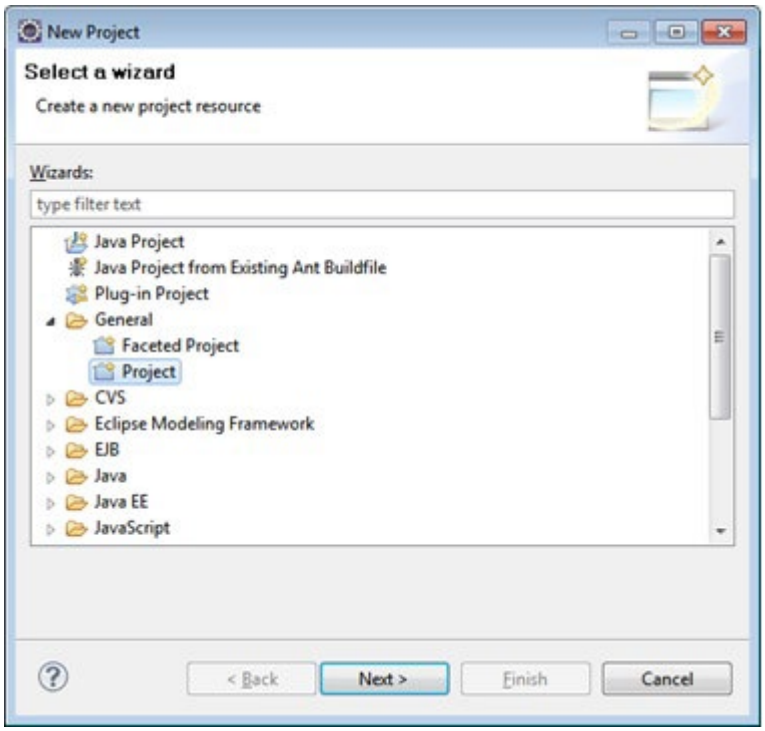
Name	Prefix	URN
control	ctl	control.services.tiplus2.misys.com
messages	msg	messages.service.ti.apps.tiplus2.misys.com
common	cmn	common.service.ti.apps.tiplus2.misys.com
custom	cust	custom.service.ti.apps.tiplus2.misys.com

The control namespace is usually used as the default in messages.

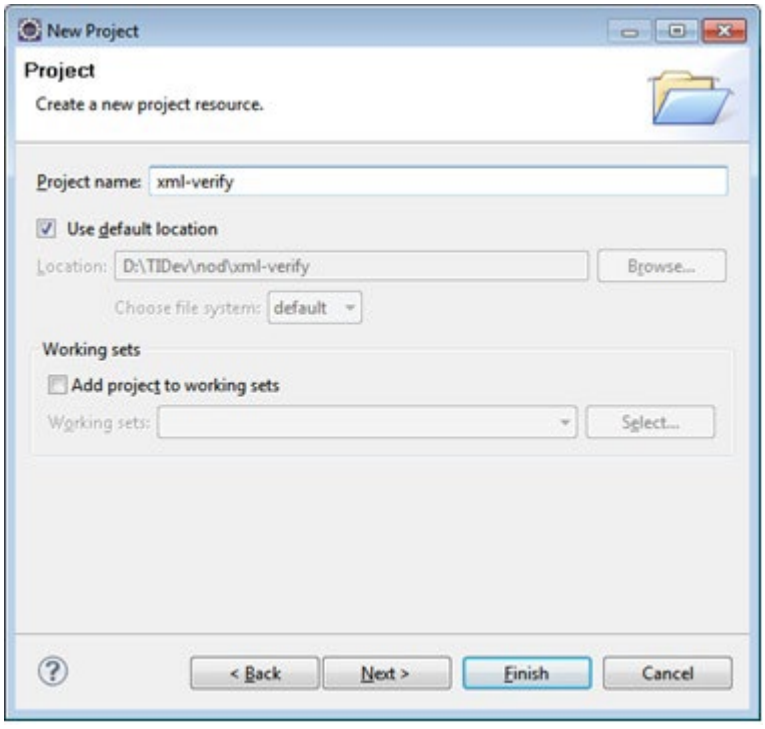
Verifying Message Formats

If you use Eclipse for development you can set up a useful verification project to keep track of message structures. You will need either the Java or JEE versions of Eclipse, these include an XML management module.

1. Create a new simple project by selecting the **File|New|Project...** menu option:

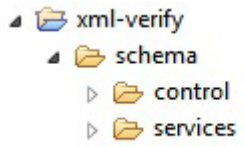


2. Click **Next**.

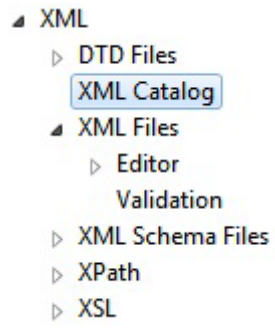


3. Enter a name for the project and click **Finish**.

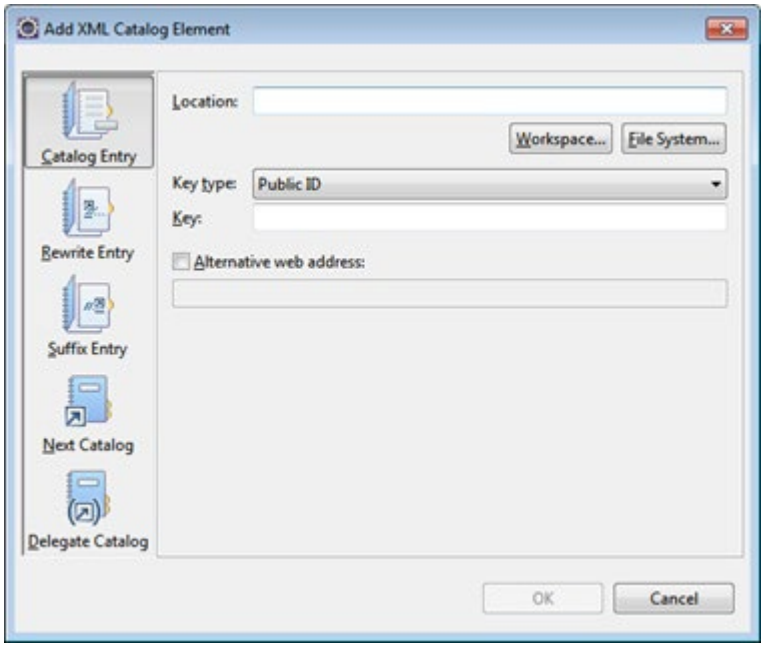
4. Create a new folder in the project called `schemas`.
5. Copy the folders holding the schemas into the `schemas` project folder:



6. Register the schemas in the XML Catalog by selecting the **Windows|Preferences** menu option.
7. Expand the **XML** node and select the **XML Catalog** entry:

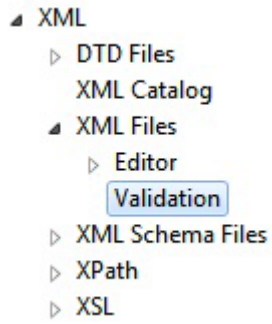


8. Click **Add** on the right-hand side of the panel:

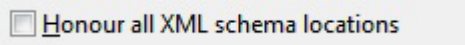


9. Using the **Workspace...** button, locate the `control.xsd` in the `/schemas/control` folder and click **OK**. Click **OK** on this dialog to add the entry.
10. Repeat the same process for the `sevices.xsd` file in the `/schemas/services` folder.

11. Select the **XML|Validation** node in the Preferences panel:



12. On the right-hand side de-select the **Honour all XML schema locations** option:



Now that the schemas have been registered, any file containing a message (e.g. from APIStubs) that is saved in this project can be validated by right-clicking on it and selecting **Validate**. This will then show where in the file the message does not conform to the schema.

This is also useful to track whether any changes to the schema will affect any interfaces defined. By having a file of each type of message covered by an interface with only those elements that are used by your interface, then any changes to the versions of the schemas (by re-importing them into the project) will show where there are any problems.

Note: Usually most changes made to schemas are backwards compatible; however in some cases this may not be possible. Make sure you refer to the Technical Release Notes for details of changes.

EJB Transport Server Component

The EJB transport server re-publishes the `ServiceProvider` interface as both a local and remote interface.

The remote interface can be used by an external application by looking up the `ejb/EnigmaServiceAccess` home interface, and creating an instance of the EJB.

To use remotely, make sure that the `com.misys.tiplus2.service.access.jar` file is included in the classpath of the client application.

Note: Since Trade Innovation is available for a number of different application servers, this jar file does not contain EJB stubs or ties. These must be generated for the appropriate application server using their own tools. See the relevant documentation for your application server.

Global JMS Transport Server

This is an optional component that acts in a similar way to the Psuedo-Synchronous transport server, that is enabled in the global application.

All service requests are processed immediately, and any replies required are placed on a nominated reply queue or the default reply queue if one is not nominated.

The deployed message-driven bean is associated with a single incoming queue, defined by the JNDI name `queue/globalIncoming`. If the message received has the JMS header field reply-to set, then the response will be sent to that queue. If a JMS correlation id has also been set, it will be set in the appropriate response.

Alternately, the `TargetReply` element in the `ServiceRequest` if specified will be used as a JNDI lookup parameter in order to locate the required response queue.

If a response is required by a message (`ReplyFormat set to FULL` or `STATUS`), and the response queue has not been identified, then a default response queue will be used defined by the `queue/globalIncomingReply` JNDI name.

JMS Components

This chapter provides details about the JMS interface components supplied.

Plain JMS Components

The plain JMS components were among the first interface components provided. They are detailed here for backwards compatibility purposes. It is recommended that the Pseudo-Synchronous JMS components be used in preference.

Transport Server

The plain JMS transport server provides a message-driven EJB that listens on a configured queue based on a name provided at application assembly time. The queue listened to is defined by the `queue/<prefix>Incoming` JNDI name.

Messages received on the queue are expected to be in the standard format described by their schema. No message response is expected.

This plain JMS transport client writes any message received to a configured queue based on a named prefix provided at application assembly time. The queue written to is defined by the `queue/<prefix>Outgoing` JNDI name. If successful, a standard `SUCCEEDED ServiceResponse` is returned.

This client should not be used for services expecting a functional response.

Pseudo-Synchronous JMS

Transport Server

This transport server provides a message-driven EJB that listens on a configured queue based on a name provided at application assembly time. The queue listened to is defined by the `queue/<prefix>Incoming` JNDI name. If the message received has the JMS header field reply-to set, then the response will be sent to that queue. If a JMS correlation id has also been set, it will be set in the appropriate response.

Alternatively, the `ReplyTarget` element in the `ServiceRequest` if specified will be used as a JNDI lookup parameter in order to locate the required response queue.

If a response is required by a message (`ReplyFormat set to FULL` or `STATUS`), and the response queue has not been identified, then a default response queue will be used defined by the `queue/<prefix>IncomingReply` JNDI name.

Messages may be transformed after they are read from the incoming queue, or before being written to the reply queue.

Transport Client

This transport client writes any message received to a configured queue based on a named prefix provided at application assembly time.

It will generate a JMS correlation id, and set it on the request message. The queue written to is defined by the `queue/<prefix>Outgoing` JNDI name. It then reads from the queue defined by the `queue/<prefix>OutgoingReply` JNDI name where the message has the same JMS correlation id as the request message.

A timeout value is configured, and if exceeded, then a standard `UNAVAILABLE ServiceResponse` is returned.

An alternate configuration option may be used to provide the same functionality as the plain JMS components, where no response is waited for.

Messages can be transformed before they are written to the outgoing queue or immediately after they are read from the reply queue.

Message Transformation

It is possible to provide simple message transformation in this configuration. Implement the interfaces `ServiceRequestTransformer` and `ServiceResponseTransformer`. Each has two methods – `transformOutgoing()` and `transformIncoming()`.

The `transformOutgoing` method is passed three parameters, the service, operation and original message, and expects the transformed message to be returned.

The `transformIncoming` method accepts a single parameter of a message in the transformed format, and expects a standard message format to be returned.

Two spring definitions are required for these implementations, one for the request transformer and once for the response transformer. The spring Ids need to be:

<prefix>.service.request.transformer

<prefix>.service.response.transformer

where `<prefix>`is the value used to define the server or client configurations.

For further information on including software in the assembly process, refer to the later chapter on creating components.

For the ServiceRequestTransformer, the transformIncoming method needs to cope with both ServiceRequests and DeferredResponses.

For the ServiceResponseTransformer, the transformOutgoing method needs to cope with both ServiceResponses and Acknowledgments.

Trade Innovation Corporate Channels Interface

This configuration uses the Pseudo-synchronous JMS mechanism as its basis.

Transport Server

Incoming TICC messages are sent via Fabric Connect (FFC). The message has `<ServiceRequest>` as the root of the message and contains `<RequestHeader>` and the gateway message type as the request body. The queue listened to is defined by the `queue/ticcIncoming` JNDI name.

Where responses to these messages are required, they will be placed on a reply queue represented by the `queue/ticcIncomingReply` JNDI name.

Transport Client

This transport client is used to send messages to the Corporate Channels Interface, which is implemented using FFC. This client retrieves the request body and uses it to process the incoming Corporate Channels messages. The queue written to is defined by the `queue/ticcOutgoing` JNDI name. If successful, a standard `SUCCEEDED ServiceResponse` is returned.

If `service.access.jms.ticc.client.timeout` is defined, the client will wait for a response from TICC which will be read from the reply queue represented by the `queue/ticcOutgoingReply` JNDI name.

MT798 Interface

This configuration uses the Pseudo-synchronous JMS mechanism as its basis.

Transport Server

Incoming MT798 messages are sent via Meridian, and so have a `<MeridianMessage>` element as the root of the message. This transport server strips off the Meridian element and passes on the content. The queue listened to is defined by the `queue/mt798Incoming` JNDI name.

Where responses to these messages are required, they will be placed on a reply queue represented by the `queue/mt798IncomingReply` JNDI name.

Transport Client

This transport client is used to send messages to the MT798 Interface, which is implemented using Meridian. This client strips some of the `RequestHeader` details from the message, and wraps the remaining details in a `MeridianMessage` element. The queue written to is defined by the `queue/mt798Outgoing` JNDI name. If successful, a standard `SUCCEEDED ServiceResponse` is returned.

Equation Interface Support

The Equation interface support is a pre-configuration of Pseudo-Synchronous JMS components.

Watch List Check Interface Support

The watch list check interface support is a pre-configuration of the plain JMS components with different JNDI names.

Service Provider Interfaces (SPIs)

Many of the Trade Innovation outgoing interactions can be in the form of REST-based SPIs (Service Provider Interfaces). These are implemented as specialised transport clients that convert a request into JSON format and call a remote REST-based URL to perform the operation. The response from that call is converted from JSON back into the expected internal response.. The REST-based URLs called are responsible for interacting with the relevant external system API.

The following SPIs are currently available to be implemented.

Service	Operation	Description
SWIFT	SwiftOut	SWIFT Out messages
Account	AvailBal	Available balance enquiry messages
Account	AccountSearch	Accounts search enquiry messages
Account	AccountDetails	Accounts details enquiry messages
Account	AccountOpen	Account open enquiry messages
GATEWAY	WLCxxxxxx	Sanction screening request (Watchlist checking) messages
Limit	Facilities	Facilities enquiry messages
Limit	FacilityDetails	Facilities details enquiry messages
Limit	Reservations	External limit reservations enquiry messages
Limit	ReservationsReversal	External limit reservations reversal enquiry messages
BackOffice	Exposure	Exposure messages
StandingSettlements	CustomerSSISearch	Customer SSI search enquiry messages
StandingSettlements	CustomerSSIDetails	Customer SSI details enquiry messages
PRINTING	PRINT_ITEM	Email items can be routed via an SPI

All the above SPIs are published on the Finastra Fusion Creator Developer Portal

https://developer.fusionfabric.cloud/explore/apis?categories=Corporate%20%26%20Commercial%20Banking%7BTrade%20and%20Supply%20Chain%20Finance%7D

Authentication for each of the SPIs uses OIDC and can use authentication via the FusionFabric.Cloud (FFDC) gateway or an on-premise provider such as KeyCloak for example.

Each SPI has its own configuration in the tiplus2.configuration.properties file and documentation on the SPI configuration can be found in the *Installation Guide – Trade Innovation*.

Note: Whenever SPI is enabled for an operation (e.g., SWIFT Out), all the related client services (except the transport client) should be removed. For example, if JMS Queue is enabled for SWIFT Out and we need to enable SWIFT Out SPI, then the SWIFT Out queue details should be removed.

The bank's API server's SSL certificate must be added to the J2EE application server's trust store for all the Bank's API HTTPs links.

Note: The configuration of sanction screening requests (watchlist check requests) – particularly with respect to the use of repeating data - requires some guidelines to be followed. These are detailed in Appendix C.

APIStubs

APIStubs is a testing utility, supplied to allow interface creators to familiarise with creating and testing messages with the system. It is supplied for use with a deployment on a single application server instance.

The tables used by the utility are detailed in *Appendix B The APIStubs Data Model*.

Note: This feature will eventually be replaced by the SDK Exerciser once parity of functionality has been achieved. For more details of the SDK Exerciser, please refer to the *SDK - Common Facilities Guide – Trade Innovation* document.

Transport Server

Based on a timer, the server checks the content of the table APISERVER, and if any new entries exist on the table, they are processed one at a time.

Entries can be added to this table using the APIStubs data editor.

Transport Client

When a service request is received, it is written to the APICLIENT table. The APIRESPONSE table is queried based on the service and operation values in order to see if a functional response is required. Each record read has a test using a regular expression that is executed on the service request. If a match is successful, then a response template held on the APIRESPONSE table is used to create a service response, substituting declared values from the request.

APICLIENT entries can be viewed and APIRESPONSE entries added using the APISTUBS data editor.

The Data Editor

The tables that are used by the APIStubs components are managed by the APIStubs data editor.

Installing the Data Editor

The data editor is a Java Swing application requiring at least Java SE 6 to be installed on the system.

To install the data editor, copy the apistubs_dataeditor.zip file from the /sdk/tools/apistubs distribution folder. Unzip the contents into a local folder.

The folder should contain:

- A jar file called com.misys.tiplus2.apistubs.dataeditor.jar
- A folder called lib

To configure the database to use, the following steps must be performed:

1. Based on which DBMS you are using, update the apistubs.hibernate.spring.xml file located in the /lib/conf/apistubs folder to import either the apistubs.db2.datasource.spring.xml file or apistubs.oracle.datasource.xml.

```
<import resource="classpath:conf/apistubs/apistubs.db2.datasource.spring.xml" />
<!--<import
resource="classpath:conf/apistubs/apistubs.oracle.datasource.spring.xml" />-->
```

Note: DB2 is the default so you only need to change the above if you are running under Oracle or MySQL.

2. Update the DBMS specific file referenced above to declare the JDBC data source details. For example if you are using DB2 then:

```
<!-- DataSource for DB2 -->
<bean id="apistubsDatasource" destroy-method="close"
          class="org.springframework.jdbc.datasource.SingleConnectionDataSource">
    <property name="driverClassName" value="com.ibm.db2.jcc.DB2Driver"/>
    <property name="url" value="jdbc:db2://localhost:50000/APISTUBS"/>
    <property name="username" value="TRADEIN1"/>
    <property name="password" value="TRADEIN1"/>
    <property name="suppressClose" value="true"/>
</bean>

<!-- SessionFactory for DB2 -->
<bean id="DB2SessionFactory" parent="AbstractSessionFactory">
    <property name="hibernateProperties">
        <props merge="true">
            <prop key="hibernate.dialect">org.hibernate.dialect.DB2Dialect</prop>
            <prop key="hibernate.default_schema">TRADEIN1</prop>
        </props>
    </property>
</bean>
```

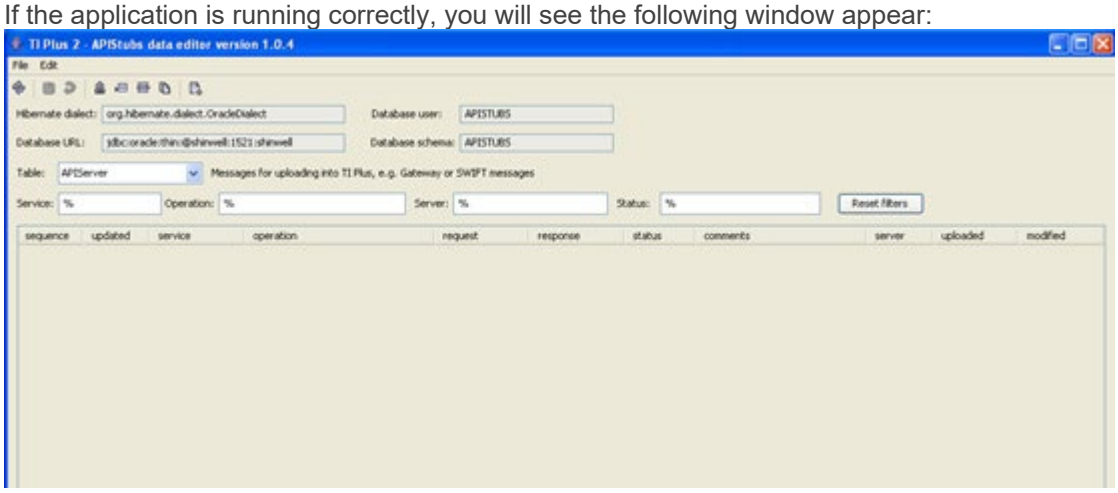
The properties to update are url, username, password and hibernate.default_schema.

3. Verify your configuration is correct by running the data editor. To do this, open a command window and enter the following command:

java –jar com.misys.tiplus2.apistubs.dataeditor.jar

Alternatively, you can double click the jar file in a Windows explorer window and as long as jar files have been associated with a Java application, the data editor should start. You won't get the command window if running in this mode.

If the application is running correctly, you will see the following window appear:



Using the Data Editor

When you run the data editor, the main window is shown:

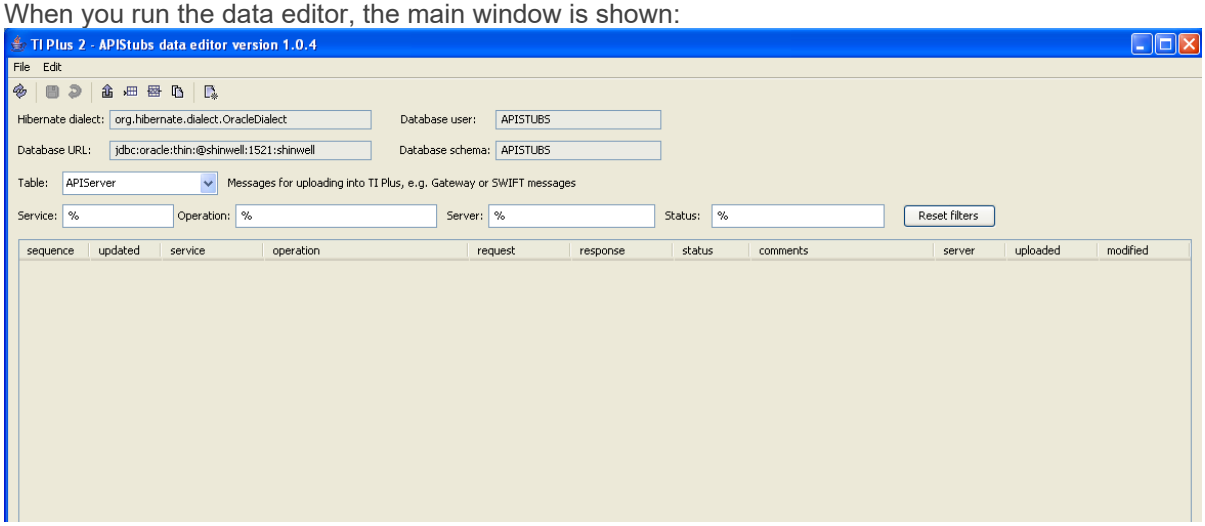


Table Selection and Filtering of Contents

The table drop down list:



allows you to select the table (`APIServer`, `APIClient` or `APIResponse`) to be displayed.

When you initially select a table, nothing will be displayed until you click the refresh  button on the toolbar. On clicking this button, the contents of the database table will be queried using the following filters:



The '%' character is the wild card character and this is what the filters default to initially and when you click the **Reset filters** button.

Adding a New Row

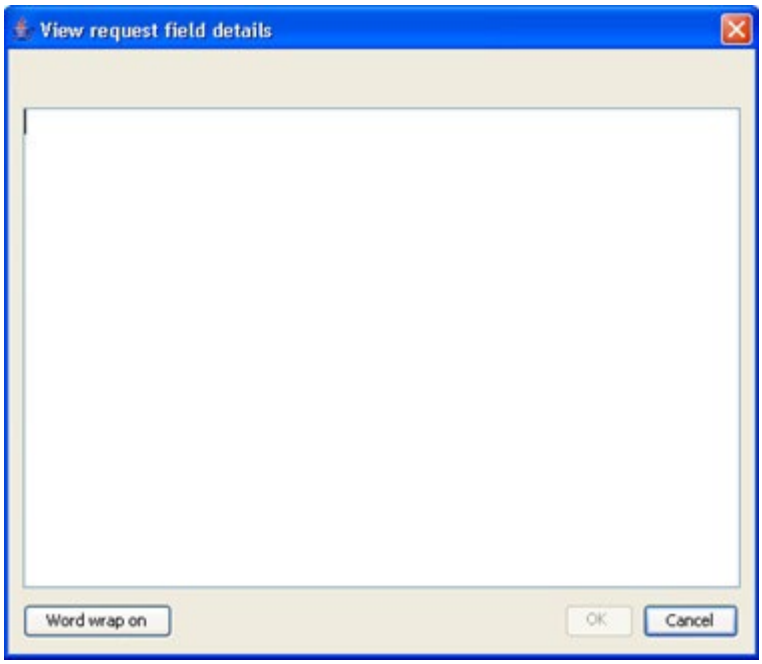
To add a new row to a table, you can select **Add Row** from the **Edit** menu or click the  button on the toolbar. This will create a blank row and the "updated" field will be set to "yes" to signify that the row needs saving.

Note: You will not be able to add a new row if there is currently an unsaved row pending.

Editing/Viewing Cell Data

Double-clicking on the cells will open an edit box to display the full contents of the cell. The utility only allows the fields required to be input by the user to be edited. All other cells will only be displayed for read only purposes.

When you double-click a cell, the following dialog box will appear to allow you to edit or view the contents:



Additionally, there is a **Word wrap** toggle button that allows you to switch the view of the text to either word wrap or no word wrap.

After you have changed the contents, you must save the data by selecting **Save** from the **File** menu or by clicking the toolbar button.

The updated column will display “yes” if the row has been updated but not yet saved and “no” otherwise. Note, only one row can be waiting to be saved at any one time.

Saving an Updated or New Row

When you create a new row or you update an existing row, the updated field of the row will be set to “yes” to indicate that the row needs to be saved to the database.

To save the row data to the database you select *Save* from the *File* menu or click the button. The “updated” field will then change to “no” to flag that the row has not been updated.

Note: The save button and **File|Save** menu item will only be enabled when there is a row that needs to be saved.

Deleting a Row

To delete a row of a table, highlight the row or rows and select **Delete** from the **Edit** menu or clicking the button on the toolbar.

Clearing a Response in the APIServer Table

You will probably want to send the same request from the APIServer table many times over. To make this easy, you can use the **APIServer->Clear Response** option from the **Edit** menu or the button to clear the responses from the highlighted tables thus causing the next poll by the APIStubs service access component of the `APIServer` table to pick up these requests again and transmit them into TI.

Upload of XML Request from File into the APIServer Table

You can upload a request XML message from a file using the **Upload** option from the **File** menu, or  button.

On making this selection, a dialog box will be displayed that will allow you to select a file containing the XML request message. This will be uploaded into the `APIServer` request field in a new row in the database ready for transmission into TI.

Copying a Row

You can create a new row from a copy of an existing row by using the copy function. This is done by highlighting the row you want to copy and selecting **Copy selected row** from the **Edit** menu use the  toolbar button.

Example Messages

Example messages are provided in the `/sdk/resources/sample/messages` folder.

Note: While these messages provide the structure of messages, they don't provide valid data – that must be provided by the user.

Creating an APIResponse

The records in the `APIResponse` tables are used to generate responses to specific requests from Trade Innovation. When a request from Trade Innovation is handled by the APIStubs service access component, it will look up in the `APIResponse` table, what response to send back.

It does this by matching the request with the following fields:

- Service
- Operation
- Test - Matching the request data with the regular expression in the TEST field

Note: The regular expression is matched using the Java regular expression support. The expression cannot contain new lines and if you want to have a catch-all expression then you can use `"\A.*\z"`. There are many web sites dedicated to regular expressions where you can learn more about what you can use.

If these match, then the defined response XML message will be returned to Trade Innovation.

If data from the request message needs to be substituted into the response message, that an XPath expression for the request can be used in the `APIResponse` message, delimited before and after by `%%`.

For example, if the correlation id of the request is required to substitute in the response, then the expression:

%%//ns1:CorrelationId%%

may be used.

The following namespaces prefixes are explicitly set for the parsed document:

- ns1 - urn:control.services.tiplus2.misys.com
- ns2 - urn:common.service.ti.apps.tiplus2.misys.com
- ns3 - urn:messages.service.ti.apps.tiplus2.misys.com
- ns4 - urn:custom.service.ti.apps.tiplus2.misys.com

If the `APIResponse` message is defined as a `DeferredResponse`, then a standard successful `ServiceResponse` will be returned, and the `DeferredResponse` written into the `APIServer` table, so it will be sent into Trade Innovation to simulate the asynchronous reply.

Developing Components

This chapter provides details on how your bank can integrate a third party product with the Trade Innovation messaging system where Trade Innovation is the client of the third party product.

Transport Client Implementation

To facilitate for example, an available balance check, a transport client implementation must be available to Trade Innovation is a Java class that implements the `TransportClient` interface.

```
package com.misys.tiplus2.messaging.controls.out;

import com.misys.tiplus2.messaging.MessagingException;

public interface TransportClient {

    String invoke(String service, String operation, String request)
                        throws MessagingException;
}
```

In the case of `Account.AvailBal`, the `service` parameter will be passed as *`Account`*, the operation `parameter` as *`AvailBal`*, and the `request` parameter is the `ServiceRequest` XML message, which will contain the `AvailBalRequest` content.

This method is then responsible for communicating with the relevant system that provides this service, and return a `ServiceResponse` XML message that contains the `AvailBalResponse` content.

If the system is unavailable, then the return message status may be set to UNAVAILABLE, which will either be presented to the user in order to handle manually, or cause the request to be resubmitted after a certain time if it is for an asynchronous message. In the case of the account balance request, this will be presented to the user.

Configuring the Transport Client

For further information on including components in the Trade Innovation assembly process, refer to the *SDK – Common Facilities Guide – Trade Innovation*.

Create a new spring configuration file called `example.client.spring.xml` in the `/local` module in the `conf/path/messaging` folder.

```
<?xml version="1.0" standalone="yes"?>
<beans xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-
beans-2.0.xsd"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns="http://www.springframework.org/schema/beans">

    <bean id='Example.AvailBal.transport.client' scope='prototype'
          class='examples.AvailBalTransportClient'/>
</beans>
```

Update the `build-tiplus2.xml` file in `/local` module to copy this configuration file as well as the resulting jar file.

To update the existing configuration to use the new transport client, use the `<springbridge>` ant task to define this alias is as follows:

```
<springbridge file=
          "${zone.folder}/conf/path/conf/messaging.bridge.spring.xml">
    <import resource="classpath:
                conf/messaging/example.client.spring.xml"/>
    <alias alias="Account.AvailBal.transport.client"
            name="Example.AvailBal.transport.client"/>
</springbridge>
```

The result of this configuration will cause all occurrences of the available balance service to use this implementation. If you need to define the implementation for a specific external system referenced in Trade Innovation, then add the name of the external system as a prefix to the alias name:

```
<springbridge file=
          "${zone.folder}/conf/path/conf/messaging.bridge.spring.xml">
<import resource="classpath:
                conf/messaging/example.client.spring.xml"/>
<alias alias="MainGL.Account.AvailBal.transport.client"
            name="Example.AvailBal.transport.client"/>
</springbridge>
```

where `MainGL` is the name of the external system.

Testing

To test this implementation, then the system would need to be configured in the following way.

Before Trade Innovation transactions release any debit postings to a customer account that is valid for funds settlement, the system will request that the account balance is checked in the core banking system to ensure that there are sufficient funds available.

If your implementation is for a particular external system, make sure that system is configured for the Available balance service on the appropriate main banking entity where you will enter a test transaction.

To test this service, set up a test transaction by opening, for example, an import letter of credit Issue event.

Ensure that the applicant is set up as a customer of your bank, with a valid unique customer identifier and customer number.

Add a single charge for the applicant and set its status to 'Take' (see the *Common Facilities User Guide - Trade Innovation* for instructions).

Edit the settlement instructions so that the charge is to be taken from an account in your bank's own books, and choose an account that belongs to the applicant that is valid for funds movements (for example, a current account). Make sure the account used has been set up with valid back office and external account numbers.

Having provided all the mandatory data for the event, click the Verify button. An available balance request service call will be issued, and Trade Innovation will wait for the response. Trade Innovation will display either the details of the response, or an error message, if the response is not returned.

Pend the transaction, rather than clicking OK to save it, so that it is available for further testing.

Available Balance Request

Your interface component will receive the `Account.AvailBalRequest` message and is responsible from converting the information into the appropriate format for the external system. The message has the following fields:

Field	What it Contains	Format
BackOfficeAccount	Back office account number.	Bank-defined format, if present.
ExternalAccount	External account number.	Bank-defined format, if present.
PostingValueDate	Value date of posting.	YYYY-MM-DD
PostingAmount	Amount in major currency units.	If the currency were GBP, then the amount 10,000.00 would be represented as 10000.00
PostingCurrency	Currency.	The three character ISO code of the currency.
DebitCredit	Whether the posting is debiting or crediting the account.	D (Debit) or C (Credit)
Customer	The unique identifier of the Trade Innovation customer who owns the account.	Alphanumeric, up to 20 characters.

Available Balance Response

Your components needs to convert the response received from the back office service into the following set of fields defined by the `Account.AvailBalResponse`:

Field	What it Contains	Format
Balance	The unsigned account balance in minor currency units.	000000001500000 (would be 15,000.00 in GBP)
Negative	Is the balance negative?	True or false.
Applies	Does the balance need to be checked for this account?	True or false.
Blocked	Is the account blocked in the back office?	True or false.
ErrorCode	An error code.	Blank (if the account exists), otherwise '2010'.
CheckedInBackOffice	Has the balance been checked in the core banking system? TI will not check if it already has been.	True or false.
ErrorMessage	Any error message from the back office.	This is unlimited for display, but only 250 characters are stored for a record of overridden warnings.
ErrorOrWarning	Treat message in ErrorMessage as an error (as opposed to warning)?	True or false

Developing a Server Interface

This section describes the steps needed to create and integrate a transport server interface for a third party product to use the services provided by Trade Innovation. No particular message will be used

here for illustration, as the main interaction will be via the `ServiceRequest/ServiceResponse` or `DeferredResponse/Acknowledgment` envelopes.

The implementation of a server interface must provide the following steps:

- message capture based on the transport protocol used (e.g. JMS queue)
- transformation of the message to the format expected by TI
- calling the process method on the relevant `ServiceProvider`
- handling the service response

Once the implementation has been defined, and the artefacts built, it is then necessary to integrate the components into the Trade Innovation application EAR file for deployment.

Message Capture

The most specific thing for a transport server component is the transport interaction itself. This guide cannot cover all of the possibilities; however it will discuss the implementation of the transport servers provided.

Transport Server	Description
JMS Server	The JMS server is a message driven bean that can be bound to a JNDI name at assembly time corresponding to the queue where messages will arrive. This uses standard J2EE facilities to provide a message-orientated interface that is clusterable. The server expects to process ServiceRequest or DeferredResponse messages, and if a reply queue is specified, to send the corresponding ServiceResponse or Acknowledgment message to it.
EJB Server	The EJB server provides a client the ability to perform a remote call into the application using RMI/IIOP. Once the client locates the EJB via JNDI, a process method can be called passing a String containing the ServiceRequest or DeferredResponse in the expected format, and will return a String containing the appropriate ServiceResponse or Acknowledgment.
APISTUBS Server	The APISTUBS server relies on polling a database table for items to process. The polling interval is provided by a timer, and all items that are available in the table are processed before the server then waits until the next interval is fired. The table has a column that represents the ServiceRequest or DeferredResponse stored as a CLOB. A second column will contain the ServiceResponse or Acknowledgment stored as a CLOB after the service is called. The poll method selects all items whose response column is empty. This implementation is straight-forward, however it does not work well in a clustered environment, as multiple server nodes will attempt to access the same table, each attempting to process the same, or similar result set.

Message Transformation

If the incoming message is not in the same format as expected by Trade Innovation, it is the responsibility of the transport server to perform not only the transformation to the relevant request format, but also from the response format if required.

This transformation could be completely programmatic, or utilise a transformation engine such as XSLT, in which case the stylesheets used would need to be included in the EAR assembly.

Calling the ServiceProvider

For transport servers, the code will need to locate and interact with a service provider. All service providers implement the `ServiceProvider` interface.

```
package com.misys.tiplus2.messaging.controls.in;

import com.misys.tiplus2.services.control.ServiceRequest;
import com.misys.tiplus2.services.control.ServiceResponse;

public interface ServiceProvider {

    String process(String input);

    @deprecated
    ServiceResponse process(ServiceRequest request);
}
```

The first process method accepts and returns Strings that contain the relevant XML messages.

The second method is used where the XML messages have been parsed into the relevant objects. Since the `ServiceProvider` must accept `ServiceRequest` and `DeferredResponse`s, this second method has been deprecated. If called, the `ServiceRequest` will be serialised into XML, and the first method called. On return the XML will be deserialised into a `ServiceResponse`.

To locate the relevant service provider, extract the name of the service required, load the spring bean whose name is `<service>.service.provider`. For details on how to locate spring beans, refer to the *SDK – Common Facilities Guide – Trade Innovation*.

Once loaded, the `process()` method can be called, passing the `ServiceRequest` as a `String` as a parameter.

Note: If the `<ReplyFormat>` element of the request is set to `NONE`, then the message will be processed asynchronously, and `null` will be returned, indicating it has been accepted for processing.

The code for the EJB server implementation is shown here as an example (the standard EJB methods and exception processing have been removed for brevity):

```
package com.misys.tiplus2.service.access.ejb;

import . . .

public class EnigmaServiceAccessBean implements javax.ejb.SessionBean {

    private static final Logger LOG = LoggerFactory
            .getLogger(EnigmaServiceAccessBean.class);

    private SessionContext sc;

    /**
     * Gets the named <code>ServiceProvider</code> defined in the spring
     * context.
     *
     * @param serviceName
     *            the name/id of the <code>ServiceProvider</code> bean.
     * @return the <code>ServiceProvider</code> defined in the spring context.
     */
    private ServiceProvider getServiceProvider(String serviceName) {
        ServiceProvider serviceProvider = null;
        if (serviceName.endsWith(".service.provider") == false) {
            serviceName += ".service.provider";
        }
        serviceProvider = (ServiceProvider) DeploymentDetails.getInstance()
                .getBean(serviceName);
        return serviceProvider;
    }

    /**
     * Retrieve the <Service> element value from XML String
     *
     * @param request XML ServiceRequest
     * @return <Service> element value
     */
    private String getServiceName(String request) throws UnrecognisedServiceName {
        int startpos = request.indexOf("Service>");
        if (startpos == -1) {
            throw new UnrecognisedServiceName(
                    "Unable to extract service name from " + request);
        }
        int endpos = request.indexOf("</", startpos);
        String s = request.substring(startpos + 8, endpos).trim();
        return s;
    }

    public String process(String request) throws UnrecognisedServiceName {
        ServiceProvider serviceProvider =
                getServiceProvider(getServiceName(request));
```

```
        String response = serviceProvider.process(request);
        return response;
    }

. . .

}
```

The code shows the implementation of the `process` method that accepts the XML request message as a string, and then loads the `ServiceProvider` by extracting the `<Service>` element value from the XML String. The `process` method is called on the `ServiceProvider` to call the relevant Trade Innovation service, returning the response directly.

The code could easily be expanded to perform some sort of generic transformation on the way in and on the way back if required.

Specific Message Processing

This chapter provides information on some of the messages and processing used by Trade Innovation to communicate with your bank's core banking systems.

Accounts

Search

The `Account.AccountSearch` interface service is used to browse the host database for accounts matching user-specified filters.

Used if the *Accounts* branch/service mapping is not set to *Internal*.

Retrieve

The `Account.AccountDetails` interface service is used to retrieve account details from the host system, using the account key selected within a transaction.

Used if the *Accounts* branch/service mapping is not set to *Internal*.

Open

The `Account.AccountOpen` interface service asks the host service to open a new account.

Used if the *Accounts* branch/service mapping is not set to *Internal*.

Balance Checking

Whenever Trade Innovation generates a posting across an account for which the account type is set as valid for funds settlement, the system will attempt to perform a balance check. This can be managed in one of two ways. Either the balance can be retrieved from your bank's core banking system, to allow Trade Innovation to check whether there are sufficient funds available for the transaction, or the core banking system can take responsibility for checking the account and providing warning or error messages if required. The same interface service - `Account.AvailBal` - is used for both methods but different information is returned in each case.

If you do not want to implement the balance retrieval or available balance checking, you need to code your interface program to return the flag set to 'balance checking not required' each time a balance retrieval request is passed.

Customers

Search

The `Customer.CustomerSearch` interface service is used to browse the host database for customers matching user-specified filters.

Used if the *Customer* branch/service mapping is not set to *Internal*.

Retrieve

The `Customer.CustomerDetails` interface service is used to retrieve customer details from the host system, using the customer key selected within a transaction.

The *`CustomersProtected`* service option also determines whether the input clerk will be able to update customer details retrieved from the host system or not.

Used if the *Customer* branch/service mapping is not set to *Internal*.

Business Support

Exchange Rates

Spot exchange rates and buy and sell exchange rates and rate codes are held as part of the currency details within Trade Innovation and on the host system.

If the *FX spot rate* branch/service mapping is defined, then during transaction processing for financing transactions, the system will retrieve spot rates from the assigned external system, rather than from the Trade Innovation database, using the `BusinessSupport.TIFXSpotRt` interface service.

Interest Rates

Base rates for interest are held as part of the currency details within Trade Innovation and on the host system.

If the *Interest rate* branch/service mapping is defined, then when a base rate code is entered during transaction processing for financing transactions, the system will retrieve the actual rate from the nominated external system, rather than from the Trade Innovation database, using the `BusinessSupport.TIBaseRate` interface service.

Payment

If the *HostPaymentRouting* service option for the *Customer settlements* branch/service mapping is set to Yes, the `BusinessSupport.PaymentRoute` interface services message is used to request payment routing information from the nominated external system.

The *HostPaymentRoutingEventField* service option identifies the field to be used to provide the value in the *PrincipalPartyCode* field on requests for payment routing details sent to the host.

If Trade Innovation is not required to generate SWIFT payments or advices directly because they are generated instead by the host system, then your bank will need to remove all the standard payment messages using the system tailoring application.

Foreign Exchange Contracts

A number of services are available relating to FX Option contracts as follow:

- If the FX Options service has been mapped to an external system for the respective main banking entity, then an FX Contracts browser will be available to search for and select from. The interface

service `BusinessSupport.FXOptionsSearch` provides the data for the browser. If this service is not mapped then the basic details of the FX contract can entered manually.

- When a foreign exchange contract is entered that uses an existing option take-up deal, the interface service `BusinessSupport.FXContracts` can be used to pass details of the intended take-up against the contract to your bank's core banking system, which checks that the contract exists; if so, additional information about the contract is returned in the response. This service is called if the service option *FXContractCheck* under the *FX deals* branch/service mapping is set to *Yes*.
- When an event involving a foreign exchange contract deal completes, `BackOffice.FxContractDrawdown` passes details of the take-up against the contract to the core banking system so that it can take up the settlement amount against it, provided that this does not involve overdrawing against the contract. This interface service is implemented by checking the *FXOptionsDealsViaAPI* system option under the *FX deals* branch/service mapping is set to 'Yes'.

Back Office

Messages associated with a BackOffice service for a particular transaction are usually sent in a `BackOffice.Batch` which is in the form of a bulk message.

Foreign Exchange Deals

Foreign exchange deals generated by Trade Innovation can be passed to the back office using the `BackOffice.FXDeal` interface service. To do this the back office Via API system parameters must be set appropriately (see the *System Tailoring User Guide - Trade Innovation* for instructions).

This interface service request includes deal type, deal directions, rate codes, and details of the amounts bought and sold together with the accounts to be used for settlement.

Foreign exchange deals generated by Trade Innovation always settle across internal suspense accounts. Within Trade Innovation the accounts to be used are identified using SP503 - FX settlement suspense. If you are using your bank's core banking system account number or external account number within Trade Innovation to map Trade Innovation accounts to the accounts in the core banking system you must ensure these are specified on the SP503 account details. The core banking system and external account numbers for the settlement suspense accounts will then be included in the deal details.

Trade Innovation supports the generation of split value date deals so the deals generated may have different purchase and sale dates. If your core banking system does not support split value date deals you must manage this in your interface program.

Postings

Whenever Trade Innovation's transaction processing generates postings they will be managed in the following way.

If the value date on the posting is before the next business date, it will be passed to the interface service as a `BackOffice.Posting` for posting as soon as the Trade Innovation transaction is authorised.

If the value date on the posting is in the future and it is across a non-contingent customer or nostro account it will be passed to the interface service as a `BackOffice.Projection` to allow the bank to accurately report future funds movements in the back office system.

Future value dated bookings and payments are normally held in Trade Innovation and when the value date is reached, they will be sent to the host as `BackOffice.Posting` messages. When the service option *PostReleaseAll* for the General ledger branch/service mapping is set to Yes, all bookings and payments are released on release of the event, regardless of the value date.

If the posting is across an internal account, such as a suspense account, Trade Innovation will not pass it to the interface service until its value date is reached.

As part of Trade Innovation's overnight processing, items which have been projected are passed to the interface service as postings when their value date is reached; items across internal accounts are also passed to the interface service as they reach their value date. The interface mechanism must therefore be available throughout the overnight cycle, as well as during the day.

Items will always be posted to the interface service in real time as the Trade Innovation transactions are authorised. If you wish to implement a batch interface service to your back office system, you will need to create batches as part of your interface program.

If you do not wish to pass projections to your back office system you must set your interface program up to mark them as processed without passing them to the back office.

In order to help identify the required account in your bank's core management system, Trade Innovation supports the use of a back office account identifier as a unique key on its account file.

As part of the credit risk management facilities, information is passed as part of the posting details allowing a posting to be matched against a previously generated reservation, if the back office system provides the limit reservation functionality and matches by postings directly.

For further details of the postings generated by Trade Innovation see the *System Tailoring User Guide - Trade Innovation* and the *Business Reference Guide - Trade Innovation.*

Charge Details

If a customer has their billing/invoicing details set to use external billing, then charge details are produced when an event is released. They are routed to the external system that provides the fees and billing service mapped to the source banking business associated with the customer.

For further details see the *System Tailoring User Guide - Trade Innovation.*

Verify

If the *ReleaseItemsValidation* main banking entity option is set to Yes, calls are made to the external system that provides the General ledger service to validate postings and make bookings and payments using the `BackOffice.Verify` interface service. When completing the Input step of a transaction Trade Innovation regenerates the postings and settlement details and calls the external system synchronously to perform validation.

Where the posting is a simple booking only the posting details are validated (to check, for example, whether any of the accounts involved are blocked. Where the posting involves a settlement both the posting details and the settlement details are validated. The validation is repeated when the event is released and, if the validation is successful, the event is released and a complete batch of postings passed to the host services asynchronously for actual booking and payment generation.

Limits

Limits covers both credit risk and facilities processing. A service mapping is made at the main banking entity level in Trade Innovation for either the Limits or Facilities service. If they are both handled by the same system, then the External system must provide both services.

Facilities

The overall message flow between Trade Innovation and a Facilities system is as follows:

- During transaction input the user requests a list of facilities – this sends a `Limit.FacilitiesRequest` to the limits system and a `Limit.FacilitiesResponse` is returned.
- The user then attaches the relevant facility to the transaction.
- On completion of the last input step, or during an early limit check step, Trade Innovation sends a `Limit.ReservationsRequest` to the limits system and a the reservation details are returned as a `Limit.ReservationsResponse.`
- The details of the reservation are then shown at the limit check step.
- If approved then a `BackOffice.Exposure` message is passed to the limits system when the transaction is released.
- If rejected then a `Limit.ReservationsReversalRequest` is sent to the limits system and a `Limit.ReservationsReversalResponse` returned. The transaction is returned to the appropriate Log or Input step.

There is also the optional `Limit.FacilityDetails` message that is used to return the current details associated with the facility for enquiry purposes only.

Credit Risk

The overall message flow between Trade Innovation and a credit risk system is as follows:

- On completion of the last input step, or during an early limit check step, Trade Innovation sends a `Limit.ReservePostingRequest` to the limits system for each liability posting that has been configure to perform a limit check and a the reservation details are returned as a `Limit.ReservePostingResponse.`
- The details of the reservation are then shown at the limit check step.
- If approved then a `BackOffice.Exposure` message is passed to the limits system when the transaction is released.
- If rejected then a `Limit.ReversePostingRequest` is sent to the facilities system and a `Limit.ReversePostingResponse` returned. The transaction is returned to the appropriate Log or Input step.

The limits system may also be requested for a money market deal reservation, using the `Limit.ReserveMM` and `Limit.ReverseMM` service operations in a similar way. However it is expected that the system receiving the money market deal will provide the details to the limits system for matching.

For further information see the System Tailoring Guide - *Trade Innovation.*

Gateway

Messages sent to corporate access systems are wholly configurable in Trade Innovation as gateway document messages. These are generated when a transaction is released. They are routed as a `GATEWAY.DOCS` service operation by default, but may be configured to be routed based on their message name as a `GATEWAY.<message name>` service operation, which is checked for first.

The structure of the message is a `ServiceRequest` with a payload based on the message definition configured in Trade Innovation. No functional response is expected for this type of message.

Notifications

Trade Innovation provides an optional notification service for aspects of the system. These are currently:

- Transaction event step status change (event-step)
- Master archive deletion (master-delete)
- Async framework Job status change (async-job)
- Async framework Step status change (async-step)

These may be activated through configuration. When notifications are generated, they will be produced with the `Notifications.All` interface service. This message may contain a number of notifications produced during the same database transaction. Each message will be for only one of the categories – these are not mixed.

It is possible to configure how the service will be processed by configuring the relevant transport client based on the category – for instance `Notifications.event-step`. If this has not been used, then `Notifications.All` will be used.

The structure of a notification message is a collection of standard fields with a payload of name-value pair fields. The content of the payload is specific for the notification type.

Event Step Status Payload

The notification sub-category is the product code.

The fields passed in the payload are:

- product-long-name
- product-short-name
- event-long-name
- event-short-name
- created-by
- ti-event-reference
- customer
- customer-name-and-address
- amount-and-currency
- team
- status
- user
- step-id
- step-description
- deadline-timestamp
- sla
- zone

Master Archive Deletion Payload

The fields passed in the payload are:

- master-reference
- main-banking-entity
- behalf-of-branch
- product-code
- principal-party
- principal-party-reference
- non-principal-party
- non-principal-party-reference
- amount-and-currency
- book-off-date
- eligible-for-deletion
- deletion-date

Job Status Change Payload

The notification sub-category is the name of the job.

The fields passed in the payload are:

- job-id
- job-description
- job-status
- job-context

The job-status field can take the following values:

Value	Description
I	Initiated
W	Waiting
X	Executing
S	Pausing
P	Paused
C	Completed
L	Failing
F	Failed
N	Cancelling
E	Cancelled

Step Status Change Payload

The notification sub-category is the name of the job.

The fields passed in the payload are:

- job-id
- job-description
- job-context
- step-id
- step-name
- step-description
- step-status
- step-criteria

The step-status field can take the following values:

Value	Description
I	Initiated
X	Executing
W	Waiting
E	Eligible
P	Completing
C	Completed
L	Failing
F	Failed
G	Failing with no retry option
T	Failed with no retry option
N	Cancelling
D	Cancelled
Y	Bypassing
B	Bypassed
H	Postponing

SWIFT

Outgoing Messages

When SWIFT messages are produced, they are transferred to the relevant system for sending on to SWIFT with the `SWIFT.SwiftOut` interface service. Ordinarily this contains a single SWIFT message, however where multi-part messages are generated (e.g. MT700/701), then all of the multi-part messages will be sent on the single `SWIFT.SwiftOut` message.

BIC Files

The `TI.SwiftAddress` interface service is available and it is assumed that a batch import of details to Trade Innovation's SWIFT address will be made using the bulk message mechanism.

TI

Controlling End of Day

The `TI.EODControl` message provides the means to control the end of day for a processing cycle. The available operations are:

- START
- PAUSE
- RESUME
- BYPASS
- RETRY
- CANCEL

The last three operations relate to a particular step within the end of day. In order to be able to provide this information, the notification facility for job and step status changes may be used.

Transaction Event Messages

Transaction event messages may be used to initiate events on a transaction, update an event in progress or identify a verification step waiting for a response.

The message formats may have the following common sections:

- Context – providing contextual details:
  - for new events (e.g. `BehalfOfBranch`)
  - for events in progress (e.g. `OurReference`, `EventReference`, `StepId`)
- Externally stored documents to attach to the event (not available for external verification response messages)
- Event notifications
- Embedded documents to store and attach to the event (not available for external verification response messages)
- The facility to reference an event step in progress is used in exchange steps and external verification steps defined in an orchestration workflow. These types of orchestration step allow a step level message document to be generated and sent to an external system. The step status is then waiting for a response. The response is returned asynchronously as an incoming `DeferredResponse`. This will contain:
  - For exchange step messages, this will contain the data that needs to be updated on the event – using the same format message as one that initiates an event (e.g. a `TI.TFILCAPP` message can be used to create or update an ILC Issue event)
  - For external verification messages, this will be a single message format (`TI.TFEXRRSP`) that will contain customised data to provide context on the response

To match a particular event step awaiting a response, the identifying fields to supply in the `<Context>` block of the message are:

- OurReference
- EventReference
- Step
- `SubsidiaryMasterPrefixAndSerial` (if this message corresponds to a subsidiary event)

So, the step message document sent to the external system must be configured to contain at least these details in order for that system to create a response to match.

If the referenced event is currently awaiting a response for the declared step in the workflow, then the message is processed. So for an exchange step, event data is updated. Whereas for an external verification step, the customised data supplied is stored. For both exchange and external review steps, based on the status of the response, the step will either complete automatically, and move on to the next eligible step, or will require user intervention.

A `DeferredResponse` provides the means to communicate errors or warnings from the external system, and match this response to the original request. The correlation id provided in the original step document messages `RequestHeader` must be provided in the incoming `DeferredHeader`.

If the status is sent in as `FAILED`, then at least one `Error` message must be supplied in the header `Details` section. If warnings are to be returned that require user intervention, then the status should be `SUCCEEDED` and `Warning` messages provided in the header details.

Any errors returned will always require user intervention to reject back to an earlier step in the workflow to in order to address the error. This then means that a response should not contain a mixture of errors and warnings.

Note: For an exchange step, even if error messages are returned, any data provided on the message will still be applied to the event. If this is not what is required, then the message must only contain the fields that identify the transaction.

Once the `DeferredResponse` has been processed, an `Acknowledgement` message is returned.

Enquiry Services

Configurable Enquiry Services

There are three configurable enquiry services. They are:

- TI.SearchMasters
- TI.RetrieveMasterDetails
- TI.RetrieveMasterSummaries

Each of these services has a request/response pair of messages. The content of the response in each case is driven by an enquiry response definition. The definitions are created using the *Enquiry responses* system tailoring function. For further information, see the *System Tailoring Guide – Trade Innovation*.

For the `TI.SearchMasters` service operation, the response message with contain a repeating set of `SearchMasterResponse` elements – one for each master that matches the search criteria. The remaining services will return a single element for the requested master.

Fixed Enquiry Services

There are two Fixed format enquiry services. They are:

- TI.RetrieveSecurityUsers – retrieve user summary information for all users currently defined – note that a dummy value must be supplied on the request message for the response to be returned – the value is not validated
- TI.RetrieveSecurityUser – retrieve full details of a named user

Note: There is also a fixed format enquiry – `TI.ZoneDashboard` – this is only relevant if you are developing a zone proxy application – See the *SDK Systems Integration Guide – Trade Innovation* for further details.

Global

If the global application service access transport server is enabled, then the following services are available:

- Global.User – allows a user to be defined in the global application.
- Global.RetrieveUser – allows the retrieval of a single named user, or all users currently defined

Note: Note that for Global.User, a user's password cannot be specified using this service – it is expected to be used where the authentication provider is a common service, not controlled or provided by Trade Innovation

Web Services

Imaging Support

To help build an interface to Trade Innovation from an imaging system, a web service is available that allows a batch of scanned images to be used to initiate an event in Trade Innovation. The web service maps the information sent on to the existing `TI.TFRCVDOC` service. This is a single event initiation service that together with the document details requires a minimal amount of information for the event to be created.

Note: The level of integration is dependent on the workflow capabilities of the imaging software used. The expectation would be that after scanning a set of related documents and storing them in a central DMS, a form would be available to fill in with the summary details associated with the documents, and the web service then called. This form would need to be developed in context with the workflow of the scanning software.

The web service is called `TICR` and implements the operation `ReceivedDocs`.

If functional errors are encountered processing the request, it will be added to a repair queue in Trade Innovation. Any technical errors that prevented the web service from calling into Trade Innovation will be returned as a (F)ailure. Items on the repair queue will be returned as (S)uccessful – with any contextual `details` in `info` entries of the details section of the `ResponseHeader`.

The URL the web service will be deployed on is:

http://<zone-server>:<port>/<ZoneAppContext>/externalMisysTI

which will effectively translate into something like:

http://ti-server:7011/tiplus2-zone/externalMisysTI

The WSDL file is supplied in the `/sdk/resources/wsdl` folder in the file `ti-service-access-wsdl.zip`. It is called `CRtoTI.wsdl` and the schemas it refers to are also in the zip file.

The use of this service is controlled through configuration parameters when assembling the application EAR files. Refer to the *Installation Guide - Trade Innovation* for more details on this.

External DMS

When Trade Innovation interacts with a DMS, it does so via a component that implements the `ContentRepository2` interface. An implementation of the `ContentRepository2` is supplied that converts the methods described on that interface into web service calls.

For information on how Trade Innovation uses an external DMS, refer to the *SDK Application Extension Guide - Trade Innovation.*

This facility assumes that an implementation of a supplied WSDL has been created, and translates all of the web service operations into interactions with the external DMS.

The WSDL is supplied in the `/sdk/resources/wsdl/` folder in the file `facilities-content-repository-wsdl.zip`. It is called `TItoCR.wsdl` and the schemas it refers to are also in the zip file.

The referenced schema file is called `messages.xsd`.

The service is called `GenericCR11` and must implement the following operations:

- UploadDoc
- AttachDoc
- UpdateDoc
- DetachDoc
- SearchDoc
- RetrieveDoc
- ViewDoc
- ReplaceDoc

Each operation has a request and response message broken down in the schema files.

The fields supplied match those described in the Java Interface. Each request message has a `RequestHeader`, and for the moment, only the `Credentials/Name` element will be populated.

Each Response has a `ResponseHeader` with a `status` and `details` field.

The `status` field can take the values of (E)rror (F)ailure, (U)navailable and (S)uccess. Error or failure will expect to have an `error` entry in the `details` section to provide further information. If unavailable, then a `warning` entry in the `details` section is expected to provide further information.

The `ReplaceDoc` operation provides a capability of replacing a document in the external DMS without changing the document identifier. This is an optional operation to implement. If it is not implemented, the system will upload a new document, detach from the old document and update the internal cross reference record where appropriate.

To indicate that the `ReplaceDoc` operation has been implemented, a configuration property must be declared for the external DMS for the system to use it. For further information, see the *Installation Guide – Trade Innovation*.

Appendix A Message Schemas

Individual messages are described in XML schema (xsd) files supplied with the software. The following tables list these messages.

Synchronous Messages

Service.Operation	Description
Account.AccountDetails	Requests details of an account
Account.AccountOpen	Requests the opening of an account
Account.AvailBal	Used to handle available balance checking
Account.BalanceCreditCheck	Used to handle balance credit checking (No longer supported)
Account.AccountSearch	Used to handle account host-based searches.
BackOffice.BatchEnquiryRequest	Internal service used to trigger creating a BackOffice.Verify request
BackOffice.ZoneStatus	Used to provide progress details for the zone during end of day
Broadcast.Dashboard	Internal service used to drive the cross-zone dashboard
Broadcast.ReallocteTransactions	Internal service used by the cross-zone dashboard
Browser.HostSearch	Used to handle host-based searches (Only used of Limits.Facilities)
Browser.DMSSearch	Used to handle DMS searches
BusinessSupport.FinPctUsce	Used to handle finance percentage and usance enquiries
BusinessSupport.FXContracts	Used to handle FX contract enquiries
BusinessSupport.FXOptionsSearch	Used to handle FX contract searches
BusinessSupport.FXRateService	Used to handle FX rate enquiries
BusinessSupport.InterestCalculation	Used to provide common interest calculation so that TI and the back end system are consistent
BusinessSupport.PaymentRoute	Used to request payment routing information
BusinessSupport.TIBaseRate	Used to handle base rate enquiries
BusinessSupport.TIFXSpotRt	Used to handle FX spot rate enquiries
Customer.CustomerDetails	Used to handle host-based requests for customer details
Customer.CustomerSearch	Used to handle host-based searches for customer details
Limits.CatCrit	Used to handle departmental limits category criteria validation.
Limit.Facilities	Used to retrieve list of facilities.
Limit.FacilityDetails	Used to retrieve an individual facility
Limit.Reservations	Used to handle facility-based limit reservation
Limit.ReservationsReversal	Used to handle facility-based limit reversal
Limit.ReserveMM	Used to handle MM deal limit reservation enquiries.
Limit.ReservePosting	Used to handle posting limit reservation
Limit.ReverseMM	Used to handle MM deal limit reversal

Service.Operation	Description
Limit.ReversePosting	Used to handle posting limit reversal
Loopback.CSVLogger	Internal service used to handle CSV message logging
Loopback.GatewayCreate	Internal service used to submit an attach documents message
Loopback.RepairMsg	Internal service used to resubmit a repaired customer gateway message
StandingSettlements.CustomerSSIDetails	Used to handle host-based requests for customer standing settlement instructions
StandingSettlements.CustomerSSISearch	Used to handle host-based searches for customer standing settlement instructions

Asynchronous Messages

Service.Operation	Description
BackOffice.Batch	Used to pass booking and payment details
BackOffice.ChargeDetails	Used to pass charge details to an external billing application
BackOffice.Exposure	Used to handle an exposure
BackOffice.FxContractDrawdown	Used to handle FX contract drawdowns
BackOffice.FXDeal	Used to handle FX deal details
BackOffice.MMDeal	Used to handle MM deal details
BackOffice.MMPrincipalIncreaseDecrease	Used to handle repayment of financing
BackOffice.Posting	Used to handle current postings
BackOffice.Projection	Used to handle future postings
Limit.Matching	No longer used
SWIFT.SwiftOut	Used to send SWIFT messages

Miscellaneous TI Messages

Service.Operation	Description
TI.EODControl	Control end of day
TI.RRFControl	Reserved for future use
TI.MessageInError	Holds the details of a message that failed schema validation
TI.OVERLAY	Internal service
TI.ZoneReallocateTranasactions	Dashboard operation for reallocating transactions

TI Enquiry Messages

Service.Operation	Description
TI.RetrieveMasterDetail	Retrieve master detail enquiry
TI.RetrieveMasterSummaries	Retrieve master summaries enquiry

Service.Operation	Description
TI.RetrieveSecurityUser	Retrieve full details for a single user
TI.RetrieveSecurityUsers	Retrieve summary details for a single user, or all users
TI.SearchMasters	Retrieve list of masters enquiry
TI.ZoneDashboard	Dashboard enquiry for a single zone enquiry

TI Transaction Messages

Service.Operation	Description
FXDealSetNegotiatedRate	For transactions at rate fixing step awaiting a negotiated rate, this message permits the rate to be provided from an external source. The rate fixing step can optionally auto complete through the system option NegotiatedFXDealManualRelease
TI.LN001	Creates a new license master record
TI.SwiftIn	Inward SWIFT messages
TI.TFATTDOC	Initiate new master/event with document attachments
TI.TFBUYCAN	Invoice buyer centric finance cancellation
TI.TFBUYFIN	Invoice buyer centric finance
TI.TFCHICOR	Inward cheque payment correspondence
TI.TFCHOCOR	Outward cheque payment correspondence
TI.TFCOLACP	Collection order accept
TI.TFCOLAMD	Collection amend
TI.TFCOLAMR	Collection amend request.
TI.TFCOLCOR	Collection order correspondence
TI.TFCOLDSR	Collection order discount
TI.TFCOLFNR	Collection order financing
TI.TFCOLNACP	Collection non-acceptance
TI.TFCOLNEW	Collection order create
TI.TFCOLPAY	Collection order payment
TI.TFCPCCRT.	Customer outward payment create
TI.TFCPHCRT	Cheque outward payment create
TI.TFCPICOR	Inward customer payment correspondence
TI.TFCPOCOR	Outward customer payment correspondence
TI.TFCPSCRT	Standing order create
TI.TFCRNADJ	Credit note adjust
TI.TFCRNCAN	Credit note cancel
TI.TFCRNLNK	Credit note link/unlink
TI.TFCRNNEW	New credit note
TI.TFCSOCOR	Standing order correspondence

Service.Operation	Description
TI.TFEGTAMD	Export guarantee amend
TI.TFEGTBCR	Guarantee received beneficiary response to cancel
TI.TFEGTBMR	Export guarantee beneficiary response to amend
TI.TFEGTCOR	Guarantee received correspondence
TI.TFEGTDPR	Export guarantee documents presented
TI.TFEGTDTO	Take-on of an export guarantee legacy transaction
TI.TFEGTNEW	Export guarantee.create
TI.TFEGTPYR	Export guarantee credit outstanding presentation
TITFELCAMD	Export letter of credit amendment
TI.TFELCAOP	Export letter of credit assignment of proceeds
TI.TFELCBCR	Export letter of credit beneficiary respond to cancel
TI.TFELCBMR	Export letter of credit beneficiary response to amend
TI.TFELCCOR	Export letter of credit correspondence
TI.TFELCDTO	Export letter of credit data take-on
TI.TFELCNEW	Export letter of credit create
TI.TFELCPYR	Export letter of credit outstanding presentation
TI.TFELCTRF	Export letter of credit transfer
TI.TFEOAAME	Export open account amendment
TI.TFEOABRC	Export open account buyer cancellation Request
TI.TFEOABRE	Export open account buyer amendment Request
TI.TFEOACAR	Export open account cancel
TI.TFEOACPR	Export open account payment request response
TI.TFEOACPY	Export open account continue payment
TI.TFEOACRE	New export open account
TI.TFEOACTO	Export open account - data take-on
TI.TFEOAPAY	Export open account payment
TI.TFEOARBC	Export seller response to buyer cancellation request
TI.TFEOARBE	Export seller response to buyer amendment request
TI.TFEOARPP	Export open account payment by response party
TI.TFEOARPY	Export open account request payment
TI.TFEOASRC	Export open account seller cancellation request
TI.TFEOASRE	Export open account seller amendment request
TI.TFEOATBC	Export TMA response to buyer cancellation request
TI.TFEOATBE	Export TMA response to buyer amendment request
TI.TFEOATSC	Export TMA response to seller cancellation request
TI.TFEOATSE	Export TMA response to seller amendment request

Service.Operation	Description
TI.TFESBAMD	Export standby amend
TI.TFESBBCR	Export standby beneficiary response to cancel
TI.TFESBBMR	Export standby beneficiary response to amend
TI.TFESBCOR	Export letter of credit standby correspondence
TI.TFESBDPR	Export letter of credit standby documents presented
TI.TFESBNEW	Export letter of credit standby create.
TI.TFESGPYR	Export standby letter of credit outstanding presentation
TI.TFEXPEVT	Expiry event
TI.TFEXRRSP	External review step response
TI.TFFACDRD	Factoring drawdown
TI.TFFACDRF	Factoring drawdown forecast
TI.TFFCBRPY	Clean BA finance repayment
TI.TFFELRPY	Export LC finance repayment
TI.TFFEORPY	Export open account finance repayment
TI.TFFIARPY	Finance invoice advance finance repayment
TI.TFFICRPY	Inward collection finance repayment
TI.TFFILRPY	Import LC finance repayment
TI.TFFINCOR	Bills financing correspondence
TI.TFFINNEW	Finance Standalone create
TI.TFFIORPY	Import open account finance repayment
TI.TFFOCRPY	Outward collection finance repayment
TI.TFFRBRPY	Reimbursements finance repayment
TI.TFFSARPY	Standalone finance repayment
TI.TFGENCNF	Upload e-banking references
TI.TFGWYACK	Delivery notification
TI.TFIBFCRE	Invoice bulk finance
TI.TFIBPAMD	Invoice bulk payment amend
TI.TFIBPCAN	Invoice bulk payment cancel
TI.TFIBPCRE	Invoice bulk payment create
TI.TFIDSRPY	Seller centric finance repayment
TI.TFIGTAMN	Import guarantee letter of credit amend
TI.TFIGTAMR	Import guarantee amend
TI.TFIGTAPP	Import guarantee create
TI.TFIGTCAN	Import guarantee cancellation
TI.TFIGTCOR	Import guarantee correspondence
TI.TIFIGTDTO	Take-on of an import guarantee legacy transaction

Service.Operation	Description
TI.TFIGTPYR	Guarantee issued payment response
TI.TFILCAMN	Import letter of credit amend
TI.TFILCAPP	Import letter of credit create
TI.TFILCCAN	Import letter of credit cancellation
TI.TFILCCOR	Import letter of credit correspondence
TI.TFILCDTO	Import letter of credit data take-on
TI.TFILCPYR	Import letter of credit outstanding claim
TI.TFINVAMD	Invoice amend
TI.TFINVCAN	Invoice cancel
TI.TFINVDIS	Invoice discount
TI.TFINVNEW	Invoice create
TI.TFINVSET	Invoice settle
TI.TFIOAAME	Import open account amendment
TI.TFIOABRC	Import open account buyer cancellation request.
TI.TFIOABRE	Import open account buyer amendment request
TI.TFIOACAR	Import open account cancel
TI.TFIOACRE	New import open account
TI.TFIOACTO	Import open account - data take-on
TI.TFIOAPBR	Import open account payment request buyer response
TI.TFIOAPGR	Import open account payment request general response
TI.TFIOAPTR	Import open account payment request TMA notification
TI.TFIOARBC	Import TMA response to buyer cancellation request
TI.TFIOARPY	Import open account request payment
TI.TFIOARSC	Import buyer response to seller cancellation request
TI.TFIOARSE	Import buyer response to seller amendment request
TI.TFIOASRC	Import open account seller cancellation request
TI.TFIOASRE	Import open account seller amendment request
TI.TFIOATBE	Import TMA response to buyer amendment request
TI.TFIOATSC	Import TMA response to seller cancellation request
TI.TFIOATSE	Import TMA response to seller amendment request
TI.TFIRFRPY	Buyer centric finance repayment
TI.TFISBAMN	Import standby amend
TI.TFISBAMR	Import standby amend
TI.TFISBAPP	Import standby letter of credit create
TI.TFISBCAN	Import standby letter of credit cancellation
TI.TFISBCOR	Import standby letter of credit correspondence

Service.Operation	Description
TI.TFISBPYR	Import standby letter of credit outstanding claim
TI.TFLICAMD	License amendment
TI.TFLICCAN	Cancel license
TI.TFLICCOR	License correspondence
TI.TFRCVDOC	Receive documents
TI.TFSELCAN	Invoice seller centric finance cancellation
TI.TFSELFIN	Invoice seller centric finance
TI.TFSHGAPP	Shipping guarantee create
TI.TFSHGCOR	Shipping guarantee correspondence
TI.TFWLCRSP	Watch-list check response

Static Data/System Tailoring/Security Messages

Service.Operation	Description
TI.Account	Account maintenance
TI.AccountBranchChange	Indicate that an account's branch will need to be changed during the next end of day.
TI.AccountOfficer	Account officer maintenance
TI.AccountType	Account type maintenance
TI.ActionRunAttributes	Action run attributes maintenance
TI.AddressType	Address type maintenance
TI.AnalysisCode	Analysis code maintenance
TI.AttachedDocumentType	Attached document type maintenance
TI.BandAmountType	Band amount maintenance
TI.BandDescription	Band description definition
TI.BandLimit	Band limit definition
TI.BaseRate	Base rate maintenance
TI.BaseRateCode	Base rate code maintenance
TI.BatchPhase	Batch phase maintenance
TI.BatchPhaseActionMapping	Phase action mapping maintenance
TI.BICPlusIBAN	BIC plus IBAN maintenance
TI.Branch	Branch maintenance
TI.BranchAddress	Branch address maintenance
TI.BranchCalendar	Branch calendar maintenance
TI.BranchServiceMap	Branch/Service mapping maintenance
TI.BusinessArea	Business area maintenance
TI.BusinessDayCalendar	Business day calendar maintenance
TI.CFG001	Internal message
TI.ChargeCode	Charge code maintenance
TI.ChargeGroup	Charge group maintenance
TI.ChargePercentageSchedule	Charge percentage schedule maintenance
TI.ChargeSchedule	Charge schedule maintenance
TI.ChargeType	Charge type maintenance
TI.ChequeNumber	Cheque number maintenance
TI.ClauseText	Clause text maintenance
TI.ClauseType	Clause type maintenance
TI.ClearingSystemCode	Clearing system code maintenance
TI.ClearingSystemID	Clearing system ID maintenance
TI.CodeDescriptions	Code descriptions maintenance

Service.Operation	Description
TI.CollateralType	Collateral type maintenance
TI.ControlAction	Custom control action maintenance
TI.CorporateAccess	Corporate access system maintenance
TI.Correspondent	Correspondent maintenance
TI.Country	Country maintenance
TI.CountryCalendar	Country calendar maintenance
TI.Currency	Currency maintenance
TI.CurrencyCalendar	Currency calendar maintenance
TI.CurrencySpotRate	Currency spot rate maintenance
TI.Customer	Customer maintenance
TI.CustomerAddress	Customer address maintenance
TI.CustomerBillingDetails	Customer billing details maintenance
TI.CustomerGraceExpiryDays	Customer grace expiry days maintenance
TI.CustomerMnmChange	Indicates that a change to a customer's mnemonic needs to be made during the next end of day.
TI.CustomError	Custom error maintenance
TI.CustomerType	Customer type maintenance
TI.DailyProcessingCycle	Daily processing cycle maintenance
TI.DealBranchChange	Indicates that a change to a deal's branch must be made during the next end of day .
TI.DealType	Deal type maintenance
TI.DefaultCountryDays	Default country days maintenance
TI.DefaultNostro	Default nostro maintenance
TI.DifferentialRate	Differential rate maintenance
TI.DocumentOfTitleDefinition	Document of title definition maintenance
TI.DocumentTemplate	Document template maintenance
TI.DocumentType	Document type maintenance
TI.DomainListEntry	Payment action maintenance
TI.DomainListMap	Payment action mapping maintenance
TI.EntityType	Entity type maintenance
TI.EventChargeMap	Event charge map maintenance
TI.EventClauseMap	Event clause map maintenance
TI.EventDocumentMap	Event document map maintenance
TI.EventFieldCheckDefinition	Event field check definition maintenance
TI.EventGroup	Event group maintenance
TI.EventSLAMap	Service level agreement maintenance
TI.EventType	Event type maintenance

Service.Operation	Description
TI.ExternalServiceDefinition	Internal service
TI.ExternalSystem	External system maintenance
TI.FinancePercentUsanceMapping	Finance percent usance mapping maintenance
TI.FXRate	FX rates maintenance
TI.FXRateCode	FX rate code maintenance
TI.GoodsCode	Goods code maintenance
TI.Group	Group maintenance
TI.GroupBaseRate	Group base rate code maintenance
TI.IBANStructure	IBAN structure maintenance
TI.IncomingEnquiryResponseDefinition	Incoming enquiry response definition maintenance
TI.Incoterms	Incoterms maintenance
TI.InstructionGroup	Instruction group maintenance
TI.InterestAccountingMap	Interest accounting map maintenance
TI.InterestSchedule	Interest schedule maintenance
TI.InterestScheduleExtended	Interest schedule maintenance for extended interest schedule
TI.InterestType	Tracer type maintenance
TI.InterestTypeMap	Interest type map maintenance
TI.InvoiceCustomer	Invoice customer maintenance
TI.Language	Language maintenance
TI.LicenseProductMapping	License product mapping maintenance
TI.LicenseTypeDefaults	License type defaults maintenance
TI.ListValue	List values maintenance
TI.LocalCustomer	Local customer maintenance
TI.LocalCustomerDefinition	Local customer definition maintenance
TI.LocalDictionary	Local dictionary maintenance
TI.Locale	Locale maintenance
TI.Log	Add an entry to the version history
TI.MasterReference	Master reference maintenance
TI.Nostro	Nostro maintenance
TI.NoteType	Note type maintenance
TI.OrchestrationDefaultStepTimes	Orchestration overall defaults maintenance
TI.OrchestrationEventStepTimes	Orchestration step completion times
TI.OrchestrationStepMap	Event orchestration step map maintenance
TI.OrchestrationTemplate	Orchestration template maintenance
TI.ParameterSet	Parameter set maintenance
TI.ParameterSetMap	Parameter set map maintenance

Service.Operation	Description
TI.PartyRole	Party role maintenance
TI.PaymentRejectCode	Payment reject code maintenance
TI.PostingNarrativeDefinition	Posting narrative definition maintenance
TI.PostingRule	Posting rule maintenance
TI.PrinterMapping	Printer mappings maintenance
TI.PrinterType	Printer types maintenance
TI.ProcessingCycleEventTypeMap	Processing cycle event type mapping maintenance
TI.ProcessingCycleMap	Processing cycle mapping maintenance
TI.Product	Product option maintenance
TI.ProductListValueMap	List values mapping maintenance
TI.ProductType	Product type maintenance
TI.ReferenceBlock	Reference block maintenance
TI.ReferenceDefinition	Reference definition maintenance
TI.ReportProfile	Report profile maintenance
TI.ReportType	Report type maintenance
TI.Reset	Gateway reset
TI.SCFBuyer	Supply chain finance buyer maintenance
TI.SCFBuyerOrSeller	Supply chain finance buyer or seller maintenance
TI.SCFProgramme	Supply chain finance programme maintenance
TI.SCFProgrammeSubType	Supply chain finance programme subtype maintenance
TI.SCFRelationship	Supply chain finance relationship maintenance
TI.SCFSellerBuyerMap	Seller buyer map maintenance
TI.SecurityGroup	Security group maintenance
TI.SecurityGroupCapabilityMap	Security group capability map maintenance
TI.SecurityUser	User maintenance
TI.SecurityUserCapabilityMap	User capability map maintenance
TI.SLADefault	Service level agreement parameterset overall defaults maintenance
TI.SLADefinition	Service level type maintenance
TI.SLAType	Service level types maintenance
TI.ST001	Performs charge schedule maintenance
TI.ST002	Performs interest schedule maintenance
TI.StaticDataAlias	No longer used
TI.StandingSettlementInstructions	Standing settlement instructions maintenance
TI.StandingSettlementInstructionsSet	Standing settlement instructions maintenance
TI.Statistics	Statistics maintenance
TI.Style	Style maintenance

Service.Operation	Description
TI.SwiftAddress	Swift address maintenance
TI.SwiftMessageMap	Swift mapping maintenance
TI.SystemDictionary	System dictionary maintenance
TI.SystemOption	Product/system options maintenance
TI.SystemParameter	System parameter maintenance
TI.Team	Team maintenance
TI.TeamBankingEntityMap	Team accounting branch maintenance
TI.TeamBranch	Team/branch maintenance
TI.TeamCustomer	Team/Customer maintenance
TI.TeamEventMap	Team/Event map maintenance
TI.TeamRole	Team/Role maintenance
TI.TeamUser	Team/User maintenance
TI.TeamUserEntity	Team/User branch maintenance
TI.TeamUserRole	Team/User role maintenance
TI.TICustomerDetails	TI customer details maintenance
TI.TISWIFTDetails	TI SWIFT details maintenance
TI.TracerSchedule	Tracer schedule maintenance
TI.TracerType	Tracer type maintenance
TI.TransactionCode	Transaction code maintenance
TI.TransferMethod	Transfer method maintenance
TI.TransliterationConversion	Transliteration conversion maintenance
TI.TransliterationExclusion	Transliteration exclusion maintenance
TI.UserRole	User role maintenance
TI.WLCDefinition	Watch list checker definition maintenance
TIBulk.Item	Bulk maintenance container

Global Messages

Service.Operation	Description
Global.RetrieveUser	Retrieve details for a single user, or all users
Global.User	User maintenance for the global application

Appendix B The APIStubs Data Model

The tables used by the APIStubs components are `APIServer`, `APIClient` and `APIResponse`. The following sections describe the layout of these tables.

Table - APIServer

Purpose

This table holds all requests that have been uploaded or are waiting to be uploaded into the Trade Innovation system. The APIStubs service access component will poll this table on a regular basis and any messages that do not have a response will be sent into Trade Innovation. The response returned by Trade Innovation will then be updated in the table so that a record of the message sent and the response given are retained in the database.

Details

Field	Description	Type
SEQUENCE (PK)	Automatically allocated sequence number	autokey
VERSION	Last modified timestamp	timestamp
SERVICE	Service name	string(255)
OPERATION	Operation name	string(255)
REQUEST	The request XML message	clob
RESPONSE	The response XML message	clob
COMMENTS	Comments	string(255)
SERVER	The server that processed the request	string(255)
UPLOADED	When the response request was update	timestamp
STATUS	The status value of the response message	string(255)

Table - APIResponse

Purpose

This table holds the responses that have been setup in the system. When a request going out of Trade Innovation is serviced by the APIStubs service access component, for non-bulk messages, the TEST field regular expression is matched with the request XML message for the given service and operation and if it matches, the response is returned.

Details

Field	Description	Type
SEQUENCE (PK)	Automatically allocated sequence number	autokey
VERSION	Last modified timestamp	timestamp
SERVICE	Service name	string(255)
OPERATION	Operation name	string(255)
RESPONSE	The response XML message	clob
BULK	Is this request part of a bulk request? (Y/N)	boolean
TEST	A regular expression used for matching the request.	string(255)
COMMENTS	Comments	string(1024)

Table - APIClient

Purpose

This table holds all of the request/response data that has been sent from Trade Innovation. i.e. every time the APIStubs adaptor responds to a request, the details of both request and response are written to this table for historical purposes.

Details

Field	Description	Type
SEQUENCE (PK)	Automatically allocated sequence number	autokey
VERSION	Last modified timestamp	timestamp
SERVICE	Service name	string(255)
OPERATION	Operation name	string(255)
REQUEST	The request XML message	clob
RESPONSE	The response XML message	clob
COMMENTS	Comments	string(255)
SERVER	The server that processed the request	string(255)
REPLIED	When the response was made	timestamp
STATUS	The status value of the response message	string(255)
PART	The part number of the request in a bulk request	string(255)

Appendix C Configuring Watchlist Checker Repeating Fields

Note: This is in the context of using the watchlist checker feature with the interfacing configured to use SPIs (Service Provider Interfaces).

Watchlist checker message definitions may vary for each product event, which can result in the production of many different messages. To provide a consistent SPI, the JSON payload is implemented as a set of field name/value pairs. Here, the field name is the configured title of the field, and the value is taken from the transaction being checked. This allows a single JSON definition to be used regardless of the transaction configuration.

This describes scenarios where there are no repeating fields. Where there are repeating fields, there are some guidelines to follow in the configuration to ensure the data is captured correctly.

Repeating Fields

Repeating fields represent an array of table entries, where each entry consists of an array of field name/value pairs. The array of table entries is referred to as a field set, and a name for the field set is derived from the configuration. A message may contain multiple sets of repeating fields.

Repeating data is represented by event fields of type Table. Each table has a list of codes representing the individual fields that make up a table entry.

While the table event field structure defines explicit sets of repeating data, it is possible to split table data into separate field sets or combine the content of multiple event tables into a single field set.

Defining Field Sets

A field set name is derived from the titles given to fields in the report message.

By using a hyphen in a field title, you can provide a two-part identifier. The first part being the field set name, and the second being the field name to use for the data. All fields with the same field set name will become the definition of a single entry in the field set.

For example, to include the Attached Documents table and use FACE and DESCR codes, you could set the field titles to Docs-FaceRef and Docs-Description. This groups the fields under the Docs field set name.

Since the field title may be specified explicitly, it is not tied to representing a single event table – though it could be if required.

This leads to the following possibilities:

- Field set name per event table – this provides a simple 1:1 relationship
- Field set name for multiple tables – this provides the ability to combine table entries from multiple tables by using the same field set name for each table. This requires the inclusion of the same number of fields from each event table with the same titles
- Different field sets containing fields from the same event table – this is like splitting the same event table into multiple field sets

Example of multiple event tables into one field set:

- “Docs” could be used as the field set name for fields from the attached documents table and the generated documents table.
- The same number of fields from each table need to be included and the same titles given to those fields.
- The result is that the entries from the first event table are generated followed by entries from the second event table which results in a field set called “Docs”, containing table entries from both.

Configuring Repeating fields for Watchlist Checking

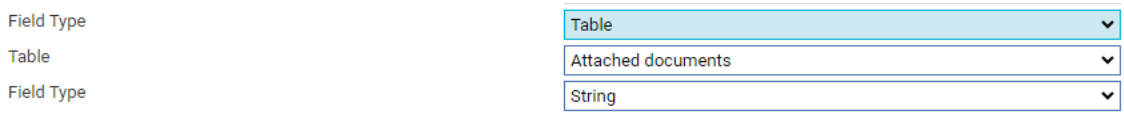
In Trade Innovation, the repeating and non-repeating fields are set in the Watchlist checker definition option for each transaction like “Import Letters of Credit – Issue”, etc.

The fields, where the field type is Table are repeating fields. So, when the fields from these tables are added, the title must be edited to have the format of <field set name>-<field name> for the repeat to be picked up at runtime. For example: “Docs-Description”, “Docs-Type”, etc.

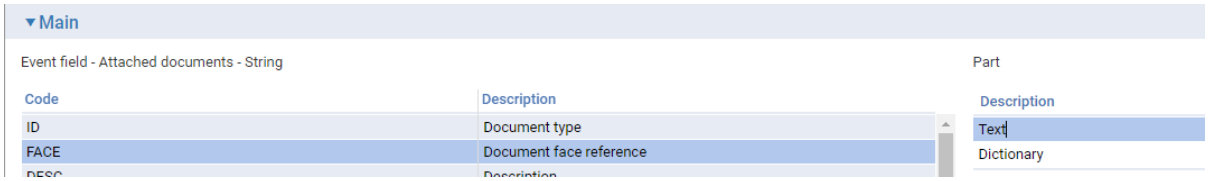
A. Adding Table Field Type to Report

To add Table Field Type:

1. Choose the Field Type as Table.
2. Choose the Table and then the Field Type for this chosen table.



3. In this section of “Event field - <for the table chosen>”, from the list, choose the field that needs to be added. Add it.



4. As displayed below, the table field type gets added.

No.	Table	Code	Title	Part	Justification	Length
66	ELCR	FACE	DocumentfacereferenceText	Text	Left	35

After adding this table field, since it is a repeating field, you need to edit it to change the title.

B. Editing the Repeating Field Title

The repeating field title can be edited. For example, the derived title “DocumentfacereferenceText” can be edited and changed to “Docs-FaceRef”, where “Docs” is the field set name and “FaceRef” is the field name.

In the same way, other fields can be added, and the title can be updated to the Report.

This is an example where there are 2 field sets (highlighted in yellow) – Docs and Date1. Docs referencing FaceRef, Description, DocType and Date with Date1 referencing StartDate and EndDate.

No.	Table	Code	Title	Part	Justification	Length
65	*	EVAM	AMOUNT	Currency and amount		
66	*	ISS	VALUEDATE	Date yyyy/mm/dd		
67	ELCR	FACE	Docs-FaceRef	Text	Left	35
68	ELCR	DESC	Docs-Description	Text	Left	35
69	ELCR	ADTY	Docs-DocType	Text	Left	35
70	ELCR	DATE	Docs-Date	Text	Left	35
71	STPA	STAD	Date1-Startdate	Date	Left	10
72	STPA	ENDD	Date1-Enddate	Date	Left	10

Once this setting is executed, click **OK** in “Watch list checker extract – Update”. This sets the fields for Watchlist checker or Sanction Screening.

Sanction Screening SPI (Watchlist checker)

Sanction Screening / Watchlist Checker uses the following confguration to generate a JSON payload.

JSON payload structure:

- Repeating field in request payload structure:

```
repeatingData []
```

```
fieldSetName
fieldSet []
       fields[] (At array level: each record)
              title (child node)
              value (child node value)
```

- Repeating field in request payload description:

  `repeatingData` will have the array of `fieldSetName` and `fieldSet`.
  `fieldSetName` will have the grouping name of the repeating field.
  `fieldSet` will have array of `fields` for each field set.
  `fields` will have array of `title` and `value` (map: for each child, array: for each record).
  `title` will have child node of the repeating field.
  `value` will have the child node's value.

- Illustration of the request payload's repeating data structure:



Limitations

When working with repeating fields, as the repeating data is detected while building the JSON payload, there is a limitation.

Repeating Fields should have consistent Field Count

- The repeating fields within each entry of a field set should have the same number of field names and values. This means that every field should have a value. If there is a field without value, then, that will be omitted and so the number of fields will differ for each table entry.
- The algorithm requires the same number of fields for each repeat.

To ensure a consistent outcome, make sure that all the repeating fields chosen have values.



The future of digital finance can start here.
Consider our planet before printing.

To find out more: visit finastra.com

© 2026 Finastra. All rights reserved

Corporate Headquarters
4 Kingdom Street
Paddington
London W2 6BD
United Kingdom

T: +44 20 3320 5000